

S4D430

Building Views in Core Data Services ABAP (CDS ABAP)

EXERCISES AND SOLUTIONS

Course Version: 20

Course Duration: 4 Hours 15 Minutes

SAP Copyrights, Trademarks and Disclaimers

© 2020 SAP SE or an SAP affiliate company. All rights reserved.

No part of this publication may be reproduced or transmitted in any form or for any purpose without the express permission of SAP SE or an SAP affiliate company.

SAP and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP SE (or an SAP affiliate company) in Germany and other countries. Please see <http://global12.sap.com/corporate-en/legal/copyright/index.epx> for additional trademark information and notices.

Some software products marketed by SAP SE and its distributors contain proprietary software components of other software vendors.

National product specifications may vary.

This course may have been machine translated and may contain grammatical errors or inaccuracies.

These materials are provided by SAP SE or an SAP affiliate company for informational purposes only, without representation or warranty of any kind, and SAP SE or its affiliated companies shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP SE or SAP affiliate company products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.

In particular, SAP SE or its affiliated companies have no obligation to pursue any course of business outlined in this document or any related presentation, or to develop or release any functionality mentioned therein. This document, or any related presentation, and SAP SE's or its affiliated companies' strategy and possible future developments, products, and/or platform directions and functionality are all subject to change and may be changed by SAP SE or its affiliated companies at any time for any reason without notice. The information in this document is not a commitment, promise, or legal obligation to deliver any material, code, or functionality. All forward-looking statements are subject to various risks and uncertainties that could cause actual results to differ materially from expectations. Readers are cautioned not to place undue reliance on these forward-looking statements, which speak only as of their dates, and they should not be relied upon in making purchasing decisions.

Typographic Conventions

American English is the standard used in this handbook.

The following typographic conventions are also used.

This information is displayed in the instructor's presentation	
Demonstration	
Procedure	
Warning or Caution	
Hint	
Related or Additional Information	
Facilitated Discussion	
User interface control	<i>Example text</i>
Window title	<i>Example text</i>

Contents

Unit 1:	Introduction
1	Exercise 1: Create a Project, a Package and a Program in ADT
4	Exercise 2: Analyze a CDS View and Use It in Open SQL
Unit 2:	Basic SQL Features in CDS Views
10	Exercise 3: Define a Simple CDS View
14	Exercise 4: Define a CDS View That Reflects a Database View
19	Exercise 5: Use Annotations in DDL Sources
Unit 3:	More SQL Features in CDS Views
24	Exercise 6: Use Expressions in a CDS View
28	Exercise 7: Use Built-in Functions in a CDS View
33	Exercise 8: Define a CDS View based on other CDS Views
39	Exercise 9: Define a CDS View with Aggregation
43	Exercise 10: Define a CDS View with a UNION statement
Unit 4:	Advanced Concepts
48	Exercise 11: Define and Use a CDS View with Input Parameter
55	Exercise 12: Define a CDS View with Associations
60	Exercise 13: Use a CDS View with Associations
66	Exercise 14: Enhance a CDS View
71	Exercise 15: Create and Use an Access Control (DCL Source)
Unit 5:	Database-Specific Features of ABAP CDS
76	Exercise 16: Define and Use a CDS Table Function
Unit 6:	Consumption of CDS Views (Outlook)
	No exercises

Create a Project, a Package and a Program in ADT

Business Example

Template:

S4D430_INTRO_T1 (Executable Program)

Solution:

S4D430_INTRO_S1 (Executable Program)

Task 1: Create a new ABAP Project in ADT

Open ABAP Development Tools. Create a project for application server S4D and add package DEVS4D430_EXERCISE as a favorite package under this project.

1. Open ABAP Development Tools, that is start Eclipse and open the ABAP perspective.
2. Create a project connected to system S4D. Use the client, user ID and password provided by your instructor.
3. Add package DEVS4D430_EXERCISE to the favorites of this project.

Task 2: Create a Package with ADT

In your project, create a new development package Z4D430_## (where ## stands for your group number) and add it to the favorite packages in your project.

1. In ADT, create a new development package Z4D430_## (where ## stands for your group number) with Software Component HOME and Application Component CA.
2. Add your package Z4D430_## to the favorite packages under your project.

Task 3: Create and Test a Program with ADT

In ADT, create a copy of executable program S4D430_INTRO_T1 and place it in your package. De-comment the source code in the program. Perform a syntax check, then activate and execute the program.

1. Create a copy of executable program S4D430_INTRO_T1 (suggested name for the copy: Z##_INTRO, where ## stands for your group number) and place the new program in your package Z4D430_##.
2. In your new program, de-comment the entire commented source code.
3. Perform a syntax check for your program.
4. Activate the program.
5. Perform a test of the program as ABAP application (SAP GUI application) from within ADT.

Create a Project, a Package and a Program in ADT

Business Example

Template:

S4D430_INTRO_T1 (Executable Program)

Solution:

S4D430_INTRO_S1 (Executable Program)

Task 1: Create a new ABAP Project in ADT

Open ABAP Development Tools. Create a project for application server S4D and add package DEVS4D430_EXERCISE as a favorite package under this project.

1. Open ABAP Development Tools, that is start Eclipse and open the ABAP perspective.
 - a) Choose *Start* → *All Programs* → *Eclipse* from the Microsoft Windows Start Menu.
 - b) If the Welcome page is open, close it or choose the *Workbench* link.
 - c) Change to the ABAP perspective using the push button in the upper right-hand corner.
2. Create a project connected to system *S4D*. Use the client, user ID and password provided by your instructor.
 - a) Right-click anywhere in the *Project Explorer* window on the left and choose *New* → *ABAP Project*.
 - b) From the system list, choose system connection *S4D SAP GUI non-SNC*. Press *Next* > followed by *Next* >.
 - c) Enter the client, user and password given to you by your instructor. Choose *Finish* .
3. Add package DEVS4D430_EXERCISE to the favorites of this project.
 - a) In the *Project Explorer*, expand the project. Right click *Favorite Packages* under your project and choose *Add a Package...*
 - b) Enter package name DEVS4D430_EXERCISE and press *OK*.

Task 2: Create a Package with ADT

In your project, create a new development package Z4D430_## (where ## stands for your group number) and add it to the favorite packages in your project.

1. In ADT, create a new development package Z4D430_## (where ## stands for your group number) with Software Component HOME and Application Component CA.
 - a) In the *Project Explorer* on the left, right-click your project and choose *New* → *ABAP Repository Object*

- b) In the dialog window *New ABAP Repository Object* expand node *Package*, choose entry *ABAP Package* and click *Next >*.
 - c) On the next dialog window, enter **Z4D430_##** (where **##** stand for your group number) and a Short Description. If necessary remove the entry in field *Superpackage*. Then choose *Next >*.
 - d) On the next dialog window, enter *Software Component* **HOME**, *Application Component* **CA** and *Transport Layer* **ZS4D**. Then choose *Next >*
 - e) On the next dialog window, select an entry from the list of requests you are involved in and choose *Next >*
 - f) Choose *Finish*.
2. Add your package Z4D430_## to the favorite packages under your project.
 - a) In the *Project Explorer*, right click *Favorite Packages* under your project and choose *Add a Package....*
 - b) Enter the name of your package and press *OK*.

Task 3: Create and Test a Program with ADT

In ADT, create a copy of executable program S4D430_INTRO_T1 and place it in your package. De-comment the source code in the program. Perform a syntax check, then activate and execute the program.

1. Create a copy of executable program S4D430_INTRO_T1 (suggested name for the copy: Z##_INTRO, where **##** stands for your group number) and place the new program in your package Z4D430_##.
 - a) In the *Project Explorer*, expand your favorite package DEVS4D430_EXERCISE. You'll find the executable program under *Source Code Library* → *Programs*
 - b) Right-click on the template program and choose *Duplicate....*
 - c) Enter the name of your package (Z4D430_##) and the name for the new program (Z##_INTRO) and choose *Next>*.
2. In your new program, de-comment the entire commented source code.
 - a) Left-click anywhere in the source code.
 - b) Hit keys **Ctrl+a** to mark the entire source code.
 - c) Hit keys **Ctrl+>** to de-comment the entire source code.
3. Perform a syntax check for your program.
 - a) Choose push button *Check ABAP Development Object* or hit keys **Ctrl+F2**.
4. Activate the program.
 - a) Choose push button *Activate* or hit keys **Ctrl+F3**.
5. Perform a test of the program as ABAP application (SAP GUI application) from within ADT.
 - a) From the menu, choose *Run* → *Run As* → *ABAP Application* or hit key **F8**.

Analyze a CDS View and Use It in Open SQL

Business Example

Template:

S4D430_INTRO_S1 (Executable Program)

Solution:

S4D430_INTRO_S2 (Executable Program)

Task 1: Search for a DDL SQL View and Navigate to the Definition

In ABAP Development Tools, search for **SQL View** S4D430_CUSTREV. Open the SQL View. Then navigate to the corresponding data definition (DDL Source).

1. In ABAP Development Tools, start a search for the ABAP repository object with name S4D430_CUSTREV.
2. Open the SQL View and navigate to the corresponding data definition (DDL Source).

What is the name of the data definition?

What is the name of the CDS View?

Task 2: Analyze the CDS View with Different ADT Tools

1. Open the DDL Source with the *Dependency Analyzer*.

Which database tables do the data come from?

2. Open the DDL Source in the *Graphical Editor*.
3. Open the *Active Annotations* view for the DDL Source.

Task 3: Open Data Preview for the CDS View

1. Use the *Data Preview* tool to display the raw data for this view.

2. Set a filter to display only flight customers located in Germany.

Task 4: Implement a SELECT Statement That Reads Data from the SQL View

Use the **SQL View** as ABAP data type and as source for a SELECT statement.



Note:

The use of SQL Views in ABAP programs is obsolete as of AS ABAP 7.50. We do it here only for didactical reasons. SAP recommends to access the CDS Views only (see next part of this exercise).

1. Create a copy of your solution of the previous exercise in your package (suggested name: Z##_OPEN_SQL, where ## is your group number). Alternatively, you can copy program S4D430_INTRO_S1.
2. Change the definition of local data type `ty_t_customers`. Replace line type SCUSTOM with **SQL View** S4D430_CUSTREV.
3. Change the SELECT statement. In the FROM clause, replace transparent table SCUSTOM with SQL View S4D430_CUSTREV.
4. Activate and test your program.

Task 5: Implement a SELECT Statement That Reads Data From the CDS View

Use the **CDS View** as ABAP data type and as source for a SELECT statement.

1. Change the definition of local data type `ty_t_customers` again. This time, replace SQL View name S4D430_CUSTREV with the corresponding **CDS View** S4D430_Customer_With_Revenue.
2. Check for possible syntax errors.

Why are SQL View and CDS View not compatible?

3. Adjust the SELECT statement to make it read data from the CDS View.

Why do you get syntax errors that requests a "@" before data object `gt_customers`?

4. Adjust the SELECT statement according to the new syntax rules as of Release 7.40: Add colons between column names and place character "@" in front of all ABAP data objects.
5. Activate and test your program.

Analyze a CDS View and Use It in Open SQL

Business Example

Template:

S4D430_INTRO_S1 (Executable Program)

Solution:

S4D430_INTRO_S2 (Executable Program)

Task 1: Search for a DDL SQL View and Navigate to the Definition

In ABAP Development Tools, search for **SQL View** S4D430_CUSTREV. Open the SQL View. Then navigate to the corresponding data definition (DDL Source).

1. In ABAP Development Tools, start a search for the ABAP repository object with name S4D430_CUSTREV.
 - a) Choose *Search* from the menu or hit keys **Ctrl+H**.
 - b) On the *ABAP Object Search* tab, enter **S4D430_CUSTREV** in field *Object name* and choose *Search*.
2. Open the SQL View and navigate to the corresponding data definition (DDL Source).
 - a) On the *Search* tab below the editor window, double-click the found repository object.
 - b) Double-click the name of the data definition in the field labeled *DDL Source*.

What is the name of the data definition?

S4D430_CUSTOMER_WITH REVENUE (all upper case)

What is the name of the CDS View?

S4d430_Customer_With_Revenue (mixture of upper and lower case)

Task 2: Analyze the CDS View with Different ADT Tools

1. Open the DDL Source with the *Dependency Analyzer*.
 - a) Right-click anywhere in the source code of the DDL Source and choose *Open With → Dependency Analyzer*.

Which database tables do the data come from?

SCUSTOM and SBOOK

2. Open the DDL Source in the *Graphical Editor*.
 - a) Right-click anywhere in the source code of the DDL Source and choose *Open With → Graphical Editor*.
3. Open the *Active Annotations* view for the DDL Source.
 - a) Right-click anywhere in the source code of the DDL Source and choose *Open With → Active Annotations*.

Task 3: Open Data Preview for the CDS View

1. Use the *Data Preview* tool to display the raw data for this view.
 - a) Right-click anywhere in the source code of the DDL Source and choose *Open With → Data Preview*.
2. Set a filter to display only flight customers located in Germany.
 - a) From the toolbar within *Data Preview*, choose *Add Filter*.
 - b) From the dropdown list, choose *Country*.
 - c) Enter filter value **DE** and choose *Enter*.

Task 4: Implement a SELECT Statement That Reads Data from the SQL View

Use the **SQL View** as ABAP data type and as source for a SELECT statement.



Note:

The use of SQL Views in ABAP programs is obsolete as of AS ABAP 7.50. We do it here only for didactical reasons. SAP recommends to access the CDS Views only (see next part of this exercise).

1. Create a copy of your solution of the previous exercise in your package (suggested name: Z##_OPEN_SQL, where ## is your group number). Alternatively, you can copy program S4D430_INTRO_S1.
 - a) See the corresponding step in the previous exercise.
2. Change the definition of local data type *ty_t_customers*. Replace line type SCUSTOM with **SQL View** S4D430_CUSTREV.
 - a) See source code extract from the model solution.
3. Change the SELECT statement. In the FROM clause, replace transparent table SCUSTOM with SQL View S4D430_CUSTREV.
 - a) See source code extract from the model solution.
4. Activate and test your program.
 - a) See the corresponding step in the previous exercise.

Task 5: Implement a SELECT Statement That Reads Data From the CDS View

Use the **CDS View** as ABAP data type and as source for a SELECT statement.

1. Change the definition of local data type `ty_t_customers` again. This time, replace SQL View name `S4D430_CUSTREV` with the corresponding **CDS View** `S4D430_Customer_With_Revenue`.
 - a) See source code extract from the model solution.
2. Check for possible syntax errors.
 - a) Open the *Problems* tab on the tabstrip below the editor.

Why are SQL View and CDS View not compatible?

The SQL View has a client field (MANDT) which is missing from the CDS View.

3. Adjust the SELECT statement to make it read data from the CDS View.
 - a) See source code extract from the model solution.

Why do you get syntax errors that requests a "@" before data object `gt_customers`?

The new syntax for Open SQL is mandatory when you use a CDS View in the FROM clause.

4. Adjust the SELECT statement according to the new syntax rules as of Release 7.40: Add colons between column names and place character "@" in front of all ABAP data objects.
 - a) See source code extract from the model solution.
5. Activate and test your program.
 - a) Perform this step as you did in the previous exercise.

Program S4D430_INTRO_S2

```
REPORT s4d430_intro_s2.

*TYPES ty_t_customers TYPE STANDARD TABLE OF scustom
*                                     WITH NON-UNIQUE KEY id.
*TYPES ty_t_customers TYPE STANDARD TABLE OF S4D430_CUSTREV
*                                     WITH NON-UNIQUE KEY id.
TYPES ty_t_customers TYPE STANDARD TABLE OF
s4d430_customer_with_revenue
                                     WITH NON-UNIQUE KEY id.

DATA gt_customers TYPE ty_t_customers.

* ALV Processing

DATA go_salv TYPE REF TO cl_salv_table.
DATA gx_excpt TYPE REF TO cx_salv_error.

* Selection Screen

PARAMETERS pa_city TYPE s_city DEFAULT 'New York' LOWER CASE.

START-OF-SELECTION.

* Data Retrieval
*****
```

```

* SELECT * FROM scustom
*         INTO TABLE gt_customers
*         WHERE city = pa_city
*         order by city name.

* Option 1: Read from the SQL View (Obsolete as off Rel. 7.50)
*****
* SELECT * FROM s4d430_custrev
*         INTO TABLE gt_customers
*         WHERE city = pa_city
*         order by city name.

* Option 2: Read from the CDS View (Recommended)
*****
      SELECT * FROM s4d430_customer_with_revenue
      INTO TABLE @gt_customers
      WHERE city = @pa_city
      ORDER BY city, name.

* output
*****
      TRY.

      cl_salv_table=>factory(
        IMPORTING
          r_salv_table   = go_salv
        CHANGING
          t_table        = gt_customers
      ).

      go_salv->display( ).

CATCH cx_salv_error INTO gx_excp.      "
      MESSAGE gx_excp TYPE 'I'.
ENDTRY.

```

Define a Simple CDS View

Business Example

Template:

none

Solution:

S4D430_SIMPLE (DDL Source)

Task 1: Create a new DDL Source

1. In your own Package, create a new DDL Source (suggested name: Z###_SIMPLE, where ## stands for your group number). Choose template *defineView* to generate the definition statement and some standard annotations.
2. If the tool did not ask for the template, go to the *templates* tab on the tabstrip below the editor and choose the template there.
3. In the respective annotation, enter a name for the SQL View that is to be generated (suggested name: Z##SIMPLE).
4. Would it be possible to specify the same name for CDS View and SQL View?

5. What is the maximum length for the name of the SQL View?

Task 2: Implement the DDL Source

1. In the FROM clause of the SELECT statement, specify database table SBOOK.
2. Edit the field list of the SELECT Statement. Start with the complete key of table SBOOK, except for the client field. Add the fields for flight class, booking price in foreign currency (with the corresponding currency key), weight of luggage (with corresponding weight unit), booking date, travel agency number, counter number and customer number.



Hint:

To enter the field names, make use of the editor's auto-completion capabilities!

Task 3: Activate the DDL Source and Test the Result

1. Perform a syntax check. Then activate the DDL Source.
2. In the *Project Explorer* window, refresh the object list for your package and locate the DB view that has been created.
3. Open the Data Preview for your CDS View.

Define a Simple CDS View

Business Example

Template:

none

Solution:

S4D430_SIMPLE (DDL Source)

Task 1: Create a new DDL Source

1. In your own Package, create a new DDL Source (suggested name: Z##_SIMPLE, where ## stands for your group number). Choose template *defineView* to generate the definition statement and some standard annotations.
 - a) In the ABAP perspective of *Eclipse*, right-click on your project and choose *New* → *Other ABAP Repository Object*.
 - b) On the next window, choose *Core Data Services* → *Data Definition*.
 - c) Enter the name of your package, and a name and short description for the new object, and choose *Next*.
 - d) If the tool asks for a template, select *defineView* and choose *Next*.
 - e) Assign the object to your transport request and choose *Finish*.
2. If the tool did not ask for the template, go to the *templates* tab on the tabstrip below the editor and choose the template there.
 - a) Go to the *templates* tab on the tabstrip below the editor.
 - b) Select template *defineView*.
3. In the respective annotation, enter a name for the SQL View that is to be generated (suggested name: Z###SIMPLE).
 - a) See source code extract from the model solution.
4. Would it be possible to specify the same name for CDS View and SQL View?

No. They need to be distinguishable when they are used, e.g. in ABAP coding.

5. What is the maximum length for the name of the SQL View?

16 Characters

Task 2: Implement the DDL Source

1. In the FROM clause of the SELECT statement, specify database table SBOOK.

- a) See source code extract from the model solution.
2. Edit the field list of the SELECT Statement. Start with the complete key of table SBOOK, except for the client field. Add the fields for flight class, booking price in foreign currency (with the corresponding currency key), weight of luggage (with corresponding weight unit), booking date, travel agency number, counter number and customer number.



Hint:

To enter the field names, make use of the editor's auto-completion capabilities!

- a) For auto-completion, press the *Ctrl* and *Space* keys at the same time.
- b) See source code extract from the model solution.

Task 3: Activate the DDL Source and Test the Result

1. Perform a syntax check. Then activate the DDL Source.
 - a) Choose *Check ABAP Development Object* or hit *Ctrl + F2*.
 - b) Choose *Activate* or hit *Ctrl + F3*.
2. In the *Project Explorer* window, refresh the object list for your package and locate the DB view that has been created.
 - a) Place the cursor on your package.
 - b) Choose *Refresh* from the context menu or hit *F5*.
 - c) Open subnode *Dictionary* → *Views*.
3. Open the Data Preview for your CDS View.
 - a) In the *Project Explorer*, right-click on your DDL Source and choose *Open With* → *Data Preview*.

DDL Source S4D430_SIMPLE

```
@AbapCatalog.sqlViewName: 'S4D430_SIMP'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Simple CDS View Definition'

define view S4D430_Simple as select
    from sbook
        {
            carrid,
            connid,
            fldate,
            bookid,
            class,
            forcuram,
            forcurkey,
            luggweight,
            wunit,
            order_date,
            agencynum,
            counter,
            customid
        }
```

Define a CDS View That Reflects a Database View

Business Example

Template:

S4D430_SIMPLE (DDL Source)

Solution:

S4D430_JOIN_AND_SELECTION (DDL Source)

Task 1: Create a new DDL Source

1. In your own Package, create a new DDL Source (suggested name: Z##_JOIN_AND_SELECTION, where ## stands for your group number). Choose template *defineViewWithJoin* to generate the definition statement and some standard annotations.
2. In the new DDL Source, provide a suitable name for the DB View (suggested name: Z##JOIN).

Task 2: Implement an inner join of tables SBOOK and SCUSTOM

1. In the definition of the CDS View, implement an inner join of tables SBOOK and SCUSTOM. Refer to the foreign key relations of table SBOOK to find the appropriate fields for the join condition. Make sure you specify aliases for the two tables.
2. Open your DDL Source Z##_SIMPLE (or the corresponding model solution S4D430_SIMPLE) and copy the entire field list to your new DDL Source. Optionally, add the alias for table SBOOK in front of the field names.
3. At the end of the list, add fields for the name and address of the customer (Name, Street, Postcode, City, Country) .



Hint:

Make use of the editor's auto-completion capabilities to enter the field names.

Task 3: Specify the key and provide aliases for some fields

1. Use prefix *key* to make sure all key fields of table SBOOK are also key fields of your view.
2. Provide an alias for fields FORCURAM and FORCURKEY (suggested alias names: AMOUNT and CURRENCY).

Task 4: Make sure that cancelled bookings are excluded from this view

1. Add a WHERE clause to read only bookings that are not cancelled.

Task 5: Activate and Test

1. Activate the DDL Source.
2. Open the Data Preview for the CDS View and ensure that the displayed data is correct.

Define a CDS View That Reflects a Database View

Business Example

Template:

S4D430_SIMPLE (DDL Source)

Solution:

S4D430_JOIN_AND_SELECTION (DDL Source)

Task 1: Create a new DDL Source

1. In your own Package, create a new DDL Source (suggested name: *Z##_JOIN_AND_SELECTION*, where *##* stands for your group number). Choose template *defineViewWithJoin* to generate the definition statement and some standard annotations.
 - a) See the corresponding step in the previous exercise.
2. In the new DDL Source, provide a suitable name for the DB View (suggested name: *Z##JOIN*).
 - a) See source code extract from the model solution.

Task 2: Implement an inner join of tables SBOOK and SCUSTOM

1. In the definition of the CDS View, implement an inner join of tables SBOOK and SCUSTOM. Refer to the foreign key relations of table SBOOK to find the appropriate fields for the join condition. Make sure you specify aliases for the two tables.
 - a) See source code extract from the model solution.
2. Open your DDL Source *Z##_SIMPLE* (or the corresponding model solution *S4D430_SIMPLE*) and copy the entire field list to your new DDL Source. Optionally, add the alias for table SBOOK in front of the field names.
 - a) Open the DDL Source in the editor window by double-clicking on it.
 - b) Select the entire field list.
 - c) Hit *Ctrl + C* to copy the source code to the clipboard.
 - d) Open the new DDL Source in the editor window.
 - e) Place the cursor between character "{" and "}" and hit *Ctrl + V*.
 - f) See source code extract from the model solution.
3. At the end of the list, add fields for the name and address of the customer (Name, Street, Postcode, City, Country) .



Hint:

Make use of the editor's auto-completion capabilities to enter the field names.

- a) See source code extract from the model solution. For the use of auto-completion, see the corresponding step in the previous exercise.

Task 3: Specify the key and provide aliases for some fields

1. Use prefix key to make sure all key fields of table SBOOK are also key fields of your view.
 - a) See source code extract from the model solution.
2. Provide an alias for fields FORCURAM and FORCURKEY (suggested alias names: AMOUNT and CURRENCY).
 - a) See source code extract from the model solution.

Task 4: Make sure that cancelled bookings are excluded from this view

1. Add a WHERE clause to read only bookings that are not cancelled.
 - a) See source code extract from the model solution.

Task 5: Activate and Test

1. Activate the DDL Source.
 - a) See the corresponding step in previous exercises.
2. Open the Data Preview for the CDS View and ensure that the displayed data is correct.
 - a) See the corresponding step in the previous exercise.

DDL Source S4D430_JOIN_AND_SELECTION

```
@AbapCatalog.sqlViewName: 'S4D430_JOIN'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Inner Join and Selection'
define view S4D430_Join_And_Selection as select
    from sbook as b inner join scustom as c
        on b.customid = c.id
    {
        key b.carriid,
        key b.connid,
        key b.fldate,
        key b.bookid,
        b.class,
        b.forcuram as amount,
        b.forcurkey as currency,
        b.luggweight,
        b.wunit,
        b.order_date,
        b.agencynum,
        b.counter,
        b.customid,
        c.name,
        c.street,
    }
```

```
        c.postcode,  
        c.city,  
        c.country  
    }  
    where cancelled <> 'X'
```

Use Annotations in DDL Sources

Business Example

Template:

S4D430_JOIN_AND_SELECTION (DDL Source)

Solution:

S4D430_ANNOTATIONS (DDL Source)

Task 1: Create a copy of a DDL Source

For the next exercise, you need a copy of your solution of the previous exercise (DDL Source Z##_JOIN_AND_SELECTION). If you did not complete the previous exercise, you can copy from DDL Source S4D430_JOIN_AND_SELECTION.



Note:

Before Release 7.51, ADT does not support the *Duplicate* function for DDL Sources. In older releases you have to create a new DDL Source, first. You then can copy and paste the source code manually.

1. Create a copy of your solution of the previous exercise or of DDL Source S4D430_JOIN_AND_SELECTION (suggested name for the DDL Source: Z##_ANNOTATIONS, where ## stands for your group number).
2. Edit the new DDL Source and replace the SQL View name (suggested name: Z##ANNO, where ## stands for your group number).

Task 2: Switch on buffering

1. In the DDL Source, add the relevant annotations to activate buffering. Set the buffering type to "Generic Buffering". Make sure that all bookings on the same flight date are buffered together.
2. How many key fields do you have to specify to achieve this?

3. Activate the DDL source.
4. Open the SQL View and check the technical settings.

Task 3: Assign a currency key and unit fields to all amount and quantity fields

1. Use the relevant field annotations to link the amount field and currency field.

2. Use the relevant field annotations to link the fields luggweight and wunit.

Task 4: Activate and Test

1. Activate the DDL Source.
2. Open the Data Preview for the CDS View and ensure that the displayed data are correct.

Use Annotations in DDL Sources

Business Example

Template:

S4D430_JOIN_AND_SELECTION (DDL Source)

Solution:

S4D430_ANNOTATIONS (DDL Source)

Task 1: Create a copy of a DDL Source

For the next exercise, you need a copy of your solution of the previous exercise (DDL Source Z##_JOIN_AND_SELECTION). If you did not complete the previous exercise, you can copy from DDL Source S4D430_JOIN_AND_SELECTION.



Note:

Before Release 7.51, ADT does not support the *Duplicate* function for DDL Sources. In older releases you have to create a new DDL Source, first. You then can copy and paste the source code manually.

1. Create a copy of your solution of the previous exercise or of DDL Source S4D430_JOIN_AND_SELECTION (suggested name for the DDL Source: Z##_ANNOTATIONS, where ## stands for your group number).
 - a) In the *Project Explorer* on the left, expand your own package. If you want to copy the template, expand favorite package DEVS4D430_EXERCISE.
 - b) Expand subnode *Core Data Services* → *Data Definitions*.
 - c) Right-click Z##_JOIN_AND_SELECTION (or S4D430_JOIN_AND_SELECTION) and choose *Duplicate*.
 - d) In field *Package*, enter the name of your own package.
 - e) In field *Name*, enter the name for the new DDL Source (suggested name: Z##_ANNOTATIONS, where ## stands for your group number) and choose *Next* >.
 - f) Choose the same transport request as before and choose *Next* >
 - g) Choose *Finish*.
2. Edit the new DDL Source and replace the SQL View name (suggested name: Z##ANNO, where ## stands for your group number).
 - a) See source code extract from the model solution.

Task 2: Switch on buffering

1. In the DDL Source, add the relevant annotations to activate buffering. Set the buffering type to "Generic Buffering". Make sure that all bookings on the same flight date are buffered together.
 - a) See source code extract from the model solution.
2. How many key fields do you have to specify to achieve this?

4 (MANDT, CARRID, CONNID, FLDATE)

3. Activate the DDL source.
 - a) See the corresponding step in previous exercises.
4. Open the SQL View and check the technical settings.
 - a) Locate the SQL View (Z##ANNO) in the *Project Explorer* and double-click on it to open the view in the ABAP Dictionary tool.
 - b) On the SAP GUI Window, choose *Menu* → *Goto* → *Technical Settings* to display the buffer settings.

Task 3: Assign a currency key and unit fields to all amount and quantity fields

1. Use the relevant field annotations to link the amount field and currency field.
 - a) See source code extract from the model solution.
2. Use the relevant field annotations to link the fields luggweight and wunit.
 - a) See source code extract from the model solution.

Task 4: Activate and Test

1. Activate the DDL Source.
 - a) See the corresponding step in the previous exercise.
2. Open the Data Preview for the CDS View and ensure that the displayed data are correct.
 - a) See the corresponding step in previous exercises.

DDL Source S4D430_ANNOTATIONS

```
@AbapCatalog.sqlViewName: 'S4D430_ANNO'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: ABAP Annotations'

@AbapCatalog.buffering.type: #GENERIC
@AbapCatalog.buffering.numberOfKeyFields: 4
@AbapCatalog.buffering.status: #ACTIVE

define view S4D430_Annotations as select
    from sbook as b inner join scustom as c
                                on b.customid = c.id
    {
        key b.carrid,
        key b.connid,
        key b.fldate,
        key b.bookid,
```

```
        b.class,  
        b.forcuram as amount  
@<Semantics.amount.currencyCode: 'CURRENCY',  
        b.forcurkey as currency  
@<Semantics.currencyCode: true,  
        b.luggweight  
@<Semantics.quantity.unitOfMeasure: 'WUNIT',  
        b.wunit  
@<Semantics.unitOfMeasure: true,  
        b.order_date,  
        b.agencynum,  
        b.counter,  
        b.customid,  
        c.name,  
        c.street,  
        c.postcode,  
        c.city,  
        c.country  
    }  
    where cancelled <> 'X'
```

Use Expressions in a CDS View

Business Example

Template:

S4D430_ANNOTATIONS (DDL Source)

Solution:

S4D430_EXPRESSIONS (DDL Source)

Task 1: Create a copy of your DDL Source

For the next exercise, you need a copy of your solution of the previous exercise (DDL Source Z##_ANNOTATIONS). If you did not complete the previous exercise, you can copy from DDL Source S4D430_ANNOTATIONS.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_EXPRESSIONS, where ## stands for your group number).
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##EXPR).

Task 2: Return readable text for the flight class

Field CLASS of table SBOOK contains a single character key for the flight class of the booking. Possible values are "Y", "C" and "F". Use the CASE expression to return longer texts for the three flight classes.

1. In the field list, replace field CLASS with a simple CASE expression to return texts "Economy", "Business" and "First" instead of values "Y", "C" and "F".

Task 3: Return the customer's discount as a factor

Field DISCOUNT of table SCUSTOM contains a customer-specific discount rate. This discount rate is stored as an integer number between 0 and 100. Implement an arithmetic expression to return the discount as a value between 0.00 and 1.00 instead.

1. Use an arithmetic expression with operator "/" to divide the discount value by 100.

Why do you have to cast the discount value to another data type?

2. Use the respective annotations to provide a label and a tooltip for the calculated discount field.

Task 4: Activate and Test

1. Activate the DDL Source.

2. Open the Data Preview for the CDS View and ensure that the displayed data are correct.

Use Expressions in a CDS View

Business Example

Template:

S4D430_ANNOTATIONS (DDL Source)

Solution:

S4D430_EXPRESSIONS (DDL Source)

Task 1: Create a copy of your DDL Source

For the next exercise, you need a copy of your solution of the previous exercise (DDL Source Z##_ANNOTATIONS). If you did not complete the previous exercise, you can copy from DDL Source S4D430_ANNOTATIONS.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_EXPRESSIONS, where ## stands for your group number).
 - a) See the corresponding step in the previous exercise.
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##EXPR).
 - a) See source code extract from the model solution.

Task 2: Return readable text for the flight class

Field CLASS of table SBOOK contains a single character key for the flight class of the booking. Possible values are "Y", "C" and "F". Use the CASE expression to return longer texts for the three flight classes.

1. In the field list, replace field CLASS with a simple CASE expression to return texts "Economy", "Business" and "First" instead of values "Y", "C" and "F".
 - a) See source code extract from the model solution.

Task 3: Return the customer's discount as a factor

Field DISCOUNT of table SCUSTOM contains a customer-specific discount rate. This discount rate is stored as an integer number between 0 and 100. Implement an arithmetic expression to return the discount as a value between 0.00 and 1.00 instead.

1. Use an arithmetic expression with operator "/" to divide the discount value by 100.

Why do you have to cast the discount value to another data type?

Because operator "/" only supports type *abap.fltp*.

- a) See source code extract from the model solution.

2. Use the respective annotations to provide a label and a tooltip for the calculated discount field.

a) See source code extract from the model solution.

Task 4: Activate and Test

1. Activate the DDL Source.
 - a) See the corresponding step in previous exercises.
2. Open the Data Preview for the CDS View and ensure that the displayed data are correct.
 - a) See the corresponding step in the previous exercise.

DDL Source S4D430_EXPRESSIONS

```
@AbapCatalog.sqlViewName: 'S4D430EXPR'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Expressions'

define view S4D430_Expressions as select
    from sbook as b inner join scustom as c
        on b.customid =
c.id
    {
        key b.carrid,
        key b.connid,
        key b.fldate,
        key b.bookid,
        case b.class
            when 'Y' then 'Economy'
            when 'C' then 'Business'
            when 'F' then 'First'
            else '???'
        end as class,
        b.forcuram as amount
    }
@<Semantics.amount.currencyCode: 'CURRENCY',
    b.forcurkey as currency
@<Semantics.currencyCode: true,
    b.luggweight
@<Semantics.quantity.unitOfMeasure: 'WUNIT',
    b.wunit
@<Semantics.unitOfMeasure: true,
    b.order_date,
    b.agencynum,
    b.counter,
    b.customid,
    c.name,
    c.street,
    c.postcode,
    c.city,
    c.country,
    @EndUserText.label: 'Discount Factor'
    @EndUserText.quickInfo: 'Customer Specific
Discount Factor'
    cast(c.discount as abap.fltp) / 100.0 as
discount
    }
where cancelled <> 'X'
```

Use Built-in Functions in a CDS View

Business Example

Template:

S4D430_EXPRESSIONS (DDL Source)

Solution:

S4D430_FUNCTIONS (DDL Source)

Task 1: Create a copy of your DDL Source

For the next exercise you need a copy of your solution of the previous exercise (DDL Source Z##_EXPRESSIONS). If you did not complete the previous exercise you can copy from DDL Source S4D430_EXPRESSIONS.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_FUNCTIONS, where ## stands for your group number).
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##FUNC).

Task 2: Use String Function `concat_with_space( )`

1. In the field list, replace the name of the flight customer (field NAME) with string function `concat_with_space( )` to concatenate the two fields FORM and NAME.



Note:

This function is available as of AS ABAP 7.50. In AS ABAP 7.40 a nested call of function `concat( )` had to be used to achieve the same result.

Task 3: Use Arithmetic Function `division( )`

1. In the calculation of the discount, replace the arithmetic expression with arithmetic function `division( )`. Make sure the result is given with two decimal places.
2. What is the advantage of using this function?

- Do you still need to cast the value of field `discount` before you can use it as input for the function?

Task 4: Perform a Currency Conversion

Use function `currency_conversion( )` so that field `AMOUNT` returns the booking price in currency USD (US Dollar).

- In the field list, replace field `LOCCURAM` with a call of built-in function `currency_conversion( )`.



Hint:

Use the editor's auto-completion capabilities to generate the function call and parameter list.

- Use fields `FORCURAM` and `FORCURKEY` as input values for parameters `source_amount` and `source_currency`.
- Set the target currency to literal value "USD."

Why is the literal "USD" itself not compatible with the functions input parameter `target_currency`?

- Choose exchange rate type "M" and do the calculation based on the exchange rate that was valid on the day of booking (field `ORDER_DATE`).
- Make sure that for all bookings, field `currency` returns the literal "USD" rather than the value of table field `FORCURKEY`.

Task 5: Calculate the Number of Days between two Dates

Calculate the number of days between the purchase of the ticket (field `ORDER_DATE`) and the actual flight date (field `FLDATE`).

- In the field list, add a new field after the field `ORDER_DATE` (suggested name: `DAYS_AHEAD`). Use function `date_diff( )` to calculate the number of days between the date fields `FLDATE` and `ORDER_DATE`.

Task 6: Activate and Test

- Activate the DDL Source.
- Open the Data Preview for the CDS View and ensure that the displayed data are correct.

Use Built-in Functions in a CDS View

Business Example

Template:

S4D430_EXPRESSIONS (DDL Source)

Solution:

S4D430_FUNCTIONS (DDL Source)

Task 1: Create a copy of your DDL Source

For the next exercise you need a copy of your solution of the previous exercise (DDL Source Z##_EXPRESSIONS). If you did not complete the previous exercise you can copy from DDL Source S4D430_EXPRESSIONS.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_FUNCTIONS, where ## stands for your group number).
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##FUNC).
 - a) See source code extract from the model solution.

Task 2: Use String Function `concat_with_space( )`

1. In the field list, replace the name of the flight customer (field NAME) with string function `concat_with_space( )` to concatenate the two fields FORM and NAME.



Note:

This function is available as of AS ABAP 7.50. In AS ABAP 7.40 a nested call of function `concat( )` had to be used to achieve the same result.

- a) See source code extract from the model solution.

Task 3: Use Arithmetic Function `division( )`

1. In the calculation of the discount, replace the arithmetic expression with arithmetic function `division( )`. Make sure the result is given with two decimal places.
 - a) See source code extract from the model solution.

2. What is the advantage of using this function?

The function supports many different numeric data types as input — not only type *abap.fltp* and it returns the result in type *abap.dec* with the possibility to set the number of decimal places for the result.

3. Do you still need to cast the value of field discount before you can use it as input for the function?

Yes, because the table field has type NUMC which is a character-like data type.

Task 4: Perform a Currency Conversion

Use function *currency_conversion()* so that field AMOUNT returns the booking price in currency USD (US Dollar).

1. In the field list, replace field LOCCURAM with a call of built-in function *currency_conversion()*.



Hint:

Use the editor's auto-completion capabilities to generate the function call and parameter list.

- a) See source code extract from the model solution.
- 2. Use fields FORCURAM and FORCURKEY as input values for parameters *source_amount* and *source_currency*.
 - a) See source code extract from the model solution.
- 3. Set the target currency to literal value "USD."

Why is the literal "USD" itself not compatible with the functions input parameter *target_currency*?

The parameter expects an input value of type *abap.cuky*.

- a) See source code extract from the model solution.
- 4. Choose exchange rate type "M" and do the calculation based on the exchange rate that was valid on the day of booking (field ORDER_DATE).
 - a) See source code extract from the model solution.
- 5. Make sure that for all bookings, field currency returns the literal "USD" rather than the value of table field FORCURKEY.
 - a) See source code extract from the model solution.

Task 5: Calculate the Number of Days between two Dates

Calculate the number of days between the purchase of the ticket (field ORDER_DATE) and the actual flight date (field FLDATE).

- 1. In the field list, add a new field after the field ORDER_DATE (suggested name: DAYS_AHEAD). Use function *days_between()* to calculate the number of days between the date fields FLDATE and ORDER_DATE

- a) See source code extract from the model solution.

Task 6: Activate and Test

1. Activate the DDL Source.
 - a) See the corresponding step in the previous exercise.
2. Open the Data Preview for the CDS View and ensure that the displayed data are correct.
 - a) See the corresponding step in previous exercises.

DDL Source S4D430_FUNCTIONS

```
@AbapCatalog.sqlViewName: 'S4D430FUNC'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Functions'
define view S4D430_Functions as select
  from sbook as b inner join scustom as c
    on b.customid = c.id
  { key b.carrid,
    key b.connid,
    key b.fldate,
    key b.bookid,
    case b.class
      when 'Y' then 'Economy'
      when 'C' then 'Business'
      when 'F' then 'First'
      else '???'
    end as class,
    currency_conversion(
      amount      => b.forcuram,
      source_currency => b.forcurkey,
      round       => 'X',
      target_currency  => cast( 'USD' as abap.cuky),
      exchange_rate_type => 'M',
      exchange_rate_date => b.order_date,
      error_handling => 'SET_TO_NULL'
    )
    as amount      @<Semantics.amount.currencyCode:'CURRENCY',
    cast('USD' as abap.cuky)
    as currency    @<Semantics.currencyCode: true,
    b.luggweight    @<Semantics.quantity.unitOfMeasure: 'WUNIT',
    b.wunit         @<Semantics.unitOfMeasure: true,
    b.order_date,
    dats_days_between(b.order_date, b.fldate) as days_ahead,
    b.agencynum,
    b.counter,
    b.customid,
    concat_with_space(c.form, c.name, 1) as name,
    // Workaround in 7.40
    // concat(concat( c.form, ' '), c.name) as name,
    c.street,
    c.postcode,
    c.city,
    c.country,
    @EndUserText.label: 'Discount Factor'
    @EndUserText.quickInfo: 'Customer Specific Discount Factor'
    division(cast(c.discount as abap.int4), 100, 2) as discount
  }
  where cancelled <> 'X'
```

Define a CDS View based on other CDS Views

Business Example

Template:

S4D430_FUNCTIONS (DDL Source)

Solution:

S4D430_BOOKING (DDL Source)

S4D430_CUSTOMER (DDL Source)

S4D430_BOOKING_AND_CUSTOMER (DDL Source)

Task 1: Define a CDS View that returns only the booking-related information

In this exercise you want to define a new CDS View that returns only the booking-related information from your solution of the previous exercise (DDL Source Z##_FUNCTIONS). If you did not complete the previous exercise you can copy from DDL Source S4D430_FUNCTIONS.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_BOOKING, where ## stands for your group number).
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##_BOOK).
3. In the FROM clause of the SELECT statement specify database table SBOOK as the data source.
4. Remove everything from the field list that is based on SCUSTOM.



Hint:

If you specify an alias for table SBOOK you can leave the rest of field list unchanged.

5. Activate the DDL Source.

Task 2: Define a CDS View that returns only the customer-related information

In this exercise you want to define a new CDS View that returns only the customer-related information from your solution of the previous exercise (DDL Source Z##_FUNCTIONS). If you did not complete the previous exercise you can copy from DDL Source S4D430_FUNCTIONS.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_CUSTOMER, where ## stands for your group number).
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##_CUST).

3. In the FROM clause of the SELECT statement specify database table SCUSTOM as data source.
4. Remove everything from the field list and the WHERE clause, that is based on SBOOK.



Hint:

If you specify an alias for table SCUSTOM you can leave the rest of field list unchanged.

5. Activate the DDL Source.

Task 3: Define a CDS View that combines data from the two CDS Views

You now want to define a new CDS View that returns exactly the same information as your solution of the previous exercise (DDL Source Z##_FUNCTIONS). But instead of reading from the database tables directly and doing the calculations itself, the new CDS View will read from two other CDS Views that encapsulate the calculations.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_BOOKING_AND_CUSTOMER, where ## stands for your group number).
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##_BOOKCUST).
3. In the new DDL Source, edit the FROM clause. Replace tables SBOOK and SCUSTOM with the two CDS Views you created earlier in this exercise (Z##_Booking and Z##_Customer).
4. Edit the field list. Remove all elements that contain expressions and functions and replace them with the corresponding fields of the CDS Views.



Note:

If you copied the WHERE clause, too, don't forget to remove it. It is superfluous in Z##_Booking_And_Customer as it is already contained in CDS View Z##_Booking.

Task 4: Activate and Test

1. Activate the DDL Sources.
2. Open the Data Preview for CDS View S4D430_BOOKING_AND_CUSTOMER and ensure that the displayed data are correct.

Define a CDS View based on other CDS Views

Business Example

Template:

S4D430_FUNCTIONS (DDL Source)

Solution:

S4D430_BOOKING (DDL Source)

S4D430_CUSTOMER (DDL Source)

S4D430_BOOKING_AND_CUSTOMER (DDL Source)

Task 1: Define a CDS View that returns only the booking-related information

In this exercise you want to define a new CDS View that returns only the booking-related information from your solution of the previous exercise (DDL Source Z##_FUNCTIONS). If you did not complete the previous exercise you can copy from DDL Source S4D430_FUNCTIONS.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_BOOKING, where ## stands for your group number).
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##BOOK).
 - a) See source code extract from the model solution.
3. In the FROM clause of the SELECT statement specify database table SBOOK as the data source.
 - a) See source code extract from the model solution.
4. Remove everything from the field list that is based on SCUSTOM.



Hint:

If you specify an alias for table SBOOK you can leave the rest of field list unchanged.

- a) See source code extract from the model solution.
5. Activate the DDL Source.
 - a) See the corresponding step in previous exercises.

DDL Source S4D430_BOOKING

```
@AbapCatalog.sqlViewName: 'S4D430_BOOK'  
@AbapCatalog.compiler.compareFilter: true
```

```

@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Flight Booking w Calculations'
define view S4D430_Booking
  as select
    from sbook as b
    { key b.carrid,
      key b.connid,
      key b.fldate,
      key b.bookid,
      case b.class
        when 'Y' then 'Economy'
        when 'C' then 'Business'
        when 'F' then 'First'
        else '???'
      end as class,
      currency_conversion(
        amount          => b.forcuram,
        source_currency => b.forcurkey,
        round            => 'X',
        target_currency => cast( 'USD' as
abap.cuky),
        exchange_rate_type => 'M',
        exchange_rate_date => b.order_date,
        error_handling => 'SET_TO_NULL'
      ) as amount

@<Semantics.amount.currencyCode: 'CURRENCY',
      cast('USD' as abap.cuky) as currency
@<Semantics.currencyCode:
true,
      b.luggweight
@<Semantics.quantity.unitOfMeasure: 'WUNIT',
      b.wunit @<Semantics.unitOfMeasure:
true,
      b.order_date,
      dats_days_between(b.order_date, b.fldate) as
days_ahead,
      b.agencynum,
      b.counter,
      b.customid
    }
  where cancelled <> 'X'

```

Task 2: Define a CDS View that returns only the customer-related information

In this exercise you want to define a new CDS View that returns only the customer-related information from your solution of the previous exercise (DDL Source Z##_FUNCTIONS). If you did not complete the previous exercise you can copy from DDL Source S4D430_FUNCTIONS.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_CUSTOMER, where ## stands for your group number).
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##_CUST).
 - a) See source code extract from the model solution.
3. In the FROM clause of the SELECT statement specify database table SCUSTOM as data source.
 - a) See source code extract from the model solution.

4. Remove everything from the field list and the WHERE clause, that is based on SBOOK.



Hint:

If you specify an alias for table SCUSTOM you can leave the rest of field list unchanged.

- a) See source code extract from the model solution.
5. Activate the DDL Source.
 - a) See the corresponding step in the previous exercise.

DDL Source S4D430_CUSTOMER

```
@AbapCatalog.sqlViewName: 'S4D430_CUST'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Flight Customer w Calculations'
define view s4d430_Customer
  as select
    from scustom as c
      { key c.id,
        concat_with_space(c.form, c.name, 1) as
name,
//      concat(concat( c.form, ' '), c.name) as name, //
Workaround in 7.40
        c.street,
        c.postcode,
        c.city,
        c.country,
        @EndUserText.label: 'Discount Factor'
        @EndUserText.quickInfo: 'Customer Specific Discount
Factor'
        division(cast(c.discount as abap.int4), 100, 2) as
discount
      }
```

Task 3: Define a CDS View that combines data from the two CDS Views

You now want to define a new CDS View that returns exactly the same information as your solution of the previous exercise (DDL Source Z##_FUNCTIONS). But instead of reading from the database tables directly and doing the calculations itself, the new CDS View will read from two other CDS Views that encapsulate the calculations.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_BOOKING_AND_CUSTOMER, where ## stands for your group number).
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##_BOOKCUST).
 - a) See source code extract from the model solution.
3. In the new DDL Source, edit the FROM clause. Replace tables SBOOK and SCUSTOM with the two CDS Views you created earlier in this exercise (Z##_Booking and Z##_Customer).
 - a) See source code extract from the model solution.
4. Edit the field list. Remove all elements that contain expressions and functions and replace them with the corresponding fields of the CDS Views.

**Note:**

If you copied the WHERE clause, too, don't forget to remove it. It is superfluous in Z##_Booking_And_Customer as it is already contained in CDS View Z##_Booking.

- a) See source code extract from the model solution.

DDL Source S4D430_BOOKING_AND_CUSTOMER

```
@AbapCatalog.sqlViewName: 'S4D430_BOOKCUST'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Join of Booking and Customer'
define view S4d430_Booking_And_Customer as select
    from S4D430_Booking as b
    inner join s4d430_customer as c
        on b.customid = c.id
    {
        key b.carriid,
        key b.connid,
        key b.fldate,
        key b.bookid,
        b.customid,
        b.amount,
        b.currency,
        b.days_ahead,
        c.name,
        c.street,
        c.postcode,
        c.city,
        c.country
    }
```

Task 4: Activate and Test

1. Activate the DDL Sources.
 - a) See the corresponding step in previous exercises.
2. Open the Data Preview for CDS View S4D430_BOOKING_AND_CUSTOMER and ensure that the displayed data are correct.
 - a) See the corresponding step in previous exercises.

Define a CDS View with Aggregation

Business Example

Template:

Solution:

S4D430_CUSTOMER_WITH_REVENUE (DDL Source)

Task 1: Create a New DDL Source

1. In your own Package, create a new DDL Source (suggested name: Z##_CUSTOMER_WITH_REVENUE, where ## stands for your group number) based on template *defineView*.
2. In the new DDL Source, provide a suitable name for the DB View (suggested name: Z##CUSTREV).

Task 2: Let the New View Read Data from Z##_Booking_And_Customer

1. In the new DDL Source, edit the FROM clause. Make the new CDS View read data from the CDS View you created in the previous exercise (Z##_Booking_And_Customer). If you did not complete the previous exercise, you can read from CDS View S4D430_Booking_And_Customer instead.
2. Edit the field list. Add field CUSTOMID as the key field and provide an alias name ID.
3. Add name and address data of the customer (name, street, postcode, city, country).

Task 3: Add Aggregations and a GROUP BY clause

1. Edit the field list. Add a sum over the converted amount. Specify an alias name (suggested value: *amount*) for the sum.
2. After the sum, add the corresponding currency code.
3. Add the required semantics annotation to link the sum to the field with the currency code.

Why is it not necessary to denote field CURRENCY as a currency field?

4. Add the average over field DAYS_AHEAD (i.e. the number of days between booking date and flight date). Specify an alias name (suggested value: *days_ahead*) for this average.
5. Add a GROUP BY clause that lists all fields from the field list that are **not** aggregate functions.



Hint:

You can use the *Quick Fix* for the respective error message to let the editor insert the correct GROUP BY clause.

Task 4: Activate and Test

1. Activate the new DDL Source.
2. Open the Data Preview for the CDS View and ensure that the displayed data are correct.

Define a CDS View with Aggregation

Business Example

Template:

Solution:

S4D430_CUSTOMER_WITH_REVENUE (DDL Source)

Task 1: Create a New DDL Source

1. In your own Package, create a new DDL Source (suggested name: Z##_CUSTOMER_WITH_REVENUE, where ## stands for your group number) based on template *defineView*.
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the DB View (suggested name: Z##CUSTREV).
 - a) See source code extract from the model solution.

Task 2: Let the New View Read Data from Z##_Booking_And_Customer

1. In the new DDL Source, edit the FROM clause. Make the new CDS View read data from the CDS View you created in the previous exercise (Z##_Booking_And_Customer). If you did not complete the previous exercise, you can read from CDS View S4D430_Booking_And_Customer instead.
 - a) See source code extract from the model solution.
2. Edit the field list. Add field CUSTOMID as the key field and provide an alias name ID.
 - a) See source code extract from the model solution.
3. Add name and address data of the customer (name, street, postcode, city, country).
 - a) See source code extract from the model solution.

Task 3: Add Aggregations and a GROUP BY clause

1. Edit the field list. Add a sum over the converted amount. Specify an alias name (suggested value: *amount*) for the sum.
 - a) See source code extract from the model solution.
2. After the sum, add the corresponding currency code.
 - a) See source code extract from the model solution.
3. Add the required semantics annotation to link the sum to the field with the currency code.
 - a) See source code extract from the model solution.

Why is it not necessary to denote field CURRENCY as a currency field?

Annotation `@Semantics.currencyCode: true` is inherited from the source CDS View.

4. Add the average over field DAYS_AHEAD (i.e. the number of days between booking date and flight date). Specify an alias name (suggested value: *days_ahead*) for this average.
 - a) See source code extract from the model solution.
5. Add a GROUP BY clause that lists all fields from the field list that are **not** aggregate functions.



Hint:

You can use the *Quick Fix* for the respective error message to let the editor insert the correct GROUP BY clause.

- a) See source code extract from the model solution.

Task 4: Activate and Test

1. Activate the new DDL Source.
 - a) See the corresponding step in previous exercises.
2. Open the Data Preview for the CDS View and ensure that the displayed data are correct.
 - a) See the corresponding step in previous exercises.

DDL Source S4D430_CUSTOMER_WITH_REVENUE

```
@AbapCatalog.sqlViewName: 'S4D430_CUSTREV'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@endUserText.label: 'Solution: Customer w Aggregated Values'
define view S4d430_Customer_With_Revenue as select
  from S4d430_Booking_And_Customer as c
  {
    key c.customid as id,
      c.name,
      c.street,
      c.postcode,
      c.city,
      c.country,
      sum(c.amount) as amount @<Semantics.amount.currencyCode:
'currency',
      c.currency,          //@<Semantics.currencyCode: true,
      avg(c.days_ahead) as days_ahead
  }
  group by
    c.customid,
    c.name,
    c.postcode,
    c.street,
    c.city,
    c.country,
    c.currency
```

Define a CDS View with a UNION statement

Business Example

Template:

Solution:

S4D430_PARTNER_WITH_REVENUE (DDL Source)

Task 1: Create a New DDL Source

1. In your own Package, create a new DDL Source (suggested name: Z##_PARTNER_WITH_REVENUE, where ## stands for your group number) based on template *defineView*.
2. In the new DDL Source, provide a suitable name for the DB View (suggested name: Z##PARTREV).

Task 2: Select Customers with Revenues

Implement a first SELECT statement to read customers with revenue

1. In the new DDL Source, edit the FROM clause. Implement a select from your CDS View Z##_Customer_With_Revenue. If you did not complete the previous exercise, you can read from CDS View S4D430_Customer_With_Revenue, instead.
2. Edit the field list. Add field id as key field.
3. Add all other fields from the basis view, i.e. name and address data, total amount with currency and field DAYS_AHEAD.

Task 3: Implement a UNION statement

Add a SELECT statement to read travel agencies with revenue.

1. After the first SELECT statement, at the very end of your DDL Source, type another SELECT statement to read from CDS View S4D430_Agency_With_Revenue.
2. Select the same fields (in the same order!) as in the first SELECT.
3. Link the two SELECT statements with keyword UNION to have the two result sets combined on the database level.

Task 4: Add a Field for the Partner Type

Add a field that helps to distinguish flight customers from travel agencies.

1. Edit the field list of the first SELECT statement. Add a literal 'C' (for Customer) between fields ID and NAME.
2. Provide an alias for the literal (suggested alias: *type*).

3. Edit the field list of the second SELECT statement. This time, add literal 'A' (for Agency).



Note:

Make sure that in both SELECT statements the literal has the same position in the field list and alias name.

Task 5: Activate and Test

1. Activate the new DDL Source.
2. Open the Data Preview for the CDS View and ensure that the displayed data are correct.

Define a CDS View with a UNION statement

Business Example

Template:

Solution:

S4D430_PARTNER_WITH_REVENUE (DDL Source)

Task 1: Create a New DDL Source

1. In your own Package, create a new DDL Source (suggested name: Z##_PARTNER_WITH_REVENUE, where ## stands for your group number) based on template *defineView*.
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the DB View (suggested name: Z##_PARTREV).
 - a) See source code extract from the model solution.

Task 2: Select Customers with Revenues

Implement a first SELECT statement to read customers with revenue

1. In the new DDL Source, edit the FROM clause. Implement a select from your CDS View Z##_Customer_With_Revenue. If you did not complete the previous exercise, you can read from CDS View S4D430_Customer_With_Revenue, instead.
 - a) See source code extract from the model solution.
2. Edit the field list. Add field id as key field.
 - a) See source code extract from the model solution.
3. Add all other fields from the basis view, i.e. name and address data, total amount with currency and field DAYS_AHEAD.
 - a) See source code extract from the model solution.

Task 3: Implement a UNION statement

Add a SELECT statement to read travel agencies with revenue.

1. After the first SELECT statement, at the very end of your DDL Source, type another SELECT statement to read from CDS View S4D430_Agency_With_Revenue.
 - a) See source code extract from the model solution.
2. Select the same fields (in the same order!) as in the first SELECT.
 - a) See source code extract from the model solution.

3. Link the two SELECT statements with keyword UNION to have the two result sets combined on the database level.
 - a) See source code extract from the model solution.

Task 4: Add a Field for the Partner Type

Add a field that helps to distinguish flight customers from travel agencies.

1. Edit the field list of the first SELECT statement. Add a literal 'C' (for Customer) between fields ID and NAME.
 - a) See source code extract from the model solution.
2. Provide an alias for the literal (suggested alias: *type*).
 - a) See source code extract from the model solution.
3. Edit the field list of the second SELECT statement. This time, add literal 'A' (for Agency).



Note:

Make sure that in both SELECT statements the literal has the same position in the field list and alias name.

- a) See source code extract from the model solution.

Task 5: Activate and Test

1. Activate the new DDL Source.
 - a) See the corresponding step in previous exercises.
2. Open the Data Preview for the CDS View and ensure that the displayed data are correct.
 - a) See the corresponding step in previous exercises.

DDL Source S4D430_PARTNER_WITH_REVENUE

```
@AbapCatalog.sqlViewName: 'S4D430PARTREV'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Union of Customers & Agencies'
define view S4D430_Partner_With_Revenue as
  select
    from S4d430_Customer_With_Revenue
    {
      key id,
      'C' as type,
      name,
      street,
      postcode,
      city,
      country,
      amount,
      currency,
      days_ahead
    }

  union

  select
    from S4d430_Agency_With_Revenue
```

```
{  
  key id,  
    'A' as type,  
    name,  
    street,  
    postcode,  
    city,  
    country,  
    amount,  
    currency,  
    days_ahead  
}
```

Define and Use a CDS View with Input Parameter

Business Example

Template:

S4D430_CUSTOMER_WITH_REVENUE (DDL Source)
S4D430_INTRO_S2 (Executable Program)

Solution:

S4D430_CUSTOMER_WITH_PARAMETER (DDL Source)
S4D430_PARAM_S1 (Executable Program)

Task 1: Define a CDS View with Input Parameter

Create a copy of your CDS View for Customers with Revenue and extend it with an input parameter that allows the user to specify a date up to which the bookings enter the calculation.

1. In your own Package, create a new DDL Source (suggested name: Z##_CUSTOMER_WITH_PARAMETER, where ## stands for your group number) based on template *defineViewWithParameters*.
2. In the new DDL Source, provide a suitable name for the DB View (suggested name: Z##CUSTREV2).
3. Provide a meaningful name for the generated input parameter (suggested name: P_) and type it with the appropriate abap data type for date fields.



Hint:

When in doubt, type **abap** . and hit *Ctrl + Space*. Search the offered data types for one with description *Date in Format YYYYMMDD*.

4. Open your DDL Source Z##_CUSTOMER_WITH_REVENUE and copy the FROM clause and the field list to the new DDL Source. If you did not complete the respective exercise, you can copy from DDL Source S4D430_CUSTOMER_WITH_REVENUE.
5. Edit the new DDL Source (i.e. the one with the input parameter). Add a WHERE clause to the SELECT statement and use the input parameter to restrict the selection to flights that took place before the given date.
6. Activate your new DDL Source and open the data preview for the new CDS View.

Task 2: Use Input Parameters in Open SQL

Use your CDS View with input parameter in Open SQL and provide a value for the parameter.

1. Create a copy of your executable program `Z##_OPEN_SQL` (suggested name: `Z##_OPEN_SQL_PARAM`, where `##` is your group number). If you did not complete the respective exercise, you can copy executable program `S4D430_INTRO_S2`, instead.
2. Change the definition of local data type `ty_t_customers`. Replace line type `S4d430_Customer_With_Revenue` with your new CDS View `Z##_Customer_With_Parameter`.
3. Change the SELECT statement. In the FROM clause, replace CDS View `S4d430_Customer_With_Revenue` with your new CDS View `Z##_Customer_With_Parameter`. Supply the CDS View's input parameter with system field `sy-datum`.
4. Activate and test your program.

Task 3: System Field as Default Value for Input Parameter

Use an annotation to link your input parameter with system field `sy-datum`.

1. Edit your DDL Source `Z##_CUSTOMER_WITH_PARAMETER`. Use annotation `@Environment.systemField` for the input parameter to use the system date as the default value.
2. Activate the DDL Source and open the data preview for the new CDS View.

Why are you still prompted to enter a value for the input parameter? And why is there no default value for the parameter?

3. Return to your executable program `Z##_OPEN_SQL_PARAM` and perform a syntax check.

Why do you suddenly get a syntax error that requires that you move the INTO clause to the very end of the SELECT statement?

4. Adjust the SELECT statement according to the new syntax rules of release 7.50: Move the INTO clause to the very end of the statement.
5. Edit your program `Z##_OPEN_SQL_PARAM`. Copy the entire SELECT statement. In the copied SELECT statement, remove the parameter passing (brackets after the name of the CDS View).
6. Perform a syntax check to verify, that now a parameter passing is not needed anymore.

Define and Use a CDS View with Input Parameter

Business Example

Template:

S4D430_CUSTOMER_WITH_REVENUE (DDL Source)
S4D430_INTRO_S2 (Executable Program)

Solution:

S4D430_CUSTOMER_WITH_PARAMETER (DDL Source)
S4D430_PARAM_S1 (Executable Program)

Task 1: Define a CDS View with Input Parameter

Create a copy of your CDS View for Customers with Revenue and extend it with an input parameter that allows the user to specify a date up to which the bookings enter the calculation.

1. In your own Package, create a new DDL Source (suggested name: Z##_CUSTOMER_WITH_PARAMETER, where ## stands for your group number) based on template *defineViewWithParameters*.
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the DB View (suggested name: Z##CUSTREV2).
 - a) See source code extract from the model solution.
3. Provide a meaningful name for the generated input parameter (suggested name: P_) and type it with the appropriate abap data type for date fields.



Hint:
When in doubt, type **abap** . and hit *Ctrl + Space*. Search the offered data types for one with description *Date in Format YYYYMMDD*.

- a) See source code extract from the model solution.
4. Open your DDL Source Z##_CUSTOMER_WITH_REVENUE and copy the FROM clause and the field list to the new DDL Source. If you did not complete the respective exercise, you can copy from DDL Source S4D430_CUSTOMER_WITH_REVENUE.
 - a) See the corresponding step in previous exercises.
 5. Edit the new DDL Source (i.e. the one with the input parameter). Add a WHERE clause to the SELECT statement and use the input parameter to restrict the selection to flights that took place before the given date.

- a) WHERE fldate < \$Parameters.P_FlightsBefore OR WHERE fldate < :P_FlightsBefore
 - b) See source code extract from the model solution.
6. Activate your new DDL Source and open the data preview for the new CDS View.
- a) See the corresponding step in the previous exercise.
 - b) When prompted to enter values for parameters, enter a valid date, e.g. today, in "technical" format YYYYMMDD.

DDL Source S4D430_CUSTOMER_WITH_PARAMETER

```
@AbapCatalog.sqlViewName: 'S4D430CUSTREV2'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Input Parameter'
define view S4d430_Customer_With_Parameter
  with parameters
    @EndUserText.label: 'Consider flights up to this date'
    P_FlightsBefore : abap.dats
  as select
    from S4d430_Booking_And_Customer as c
  {
    key c.customid as id,
      c.name,
      c.street,
      c.postcode,
      c.city,
      c.country,
      sum(c.amount) as amount @<Semantics.amount.currencyCode:
'currency',
      c.currency,          //<@Semantics.currencyCode: true,
      avg(c.days_ahead) as days_ahead
  }
  where fldate < $parameters.P_FlightsBefore
  group by
    c.customid,
    c.name,
    c.postcode,
    c.street,
    c.city,
    c.country,
    c.currency
```

Task 2: Use Input Parameters in Open SQL

Use your CDS View with input parameter in Open SQL and provide a value for the parameter.

1. Create a copy of your executable program Z##_OPEN_SQL (suggested name: Z##_OPEN_SQL_PARAM, where ## is your group number). If you did not complete the respective exercise, you can copy executable program S4D430_INTRO_S2, instead.
 - a) See the corresponding step in previous exercises.
2. Change the definition of local data type *ty_t_customers*. Replace line type S4d430_Customer_With_Revenue with your new CDS View Z##_Customer_With_Parameter.
 - a) See source code extract from the model solution.
3. Change the SELECT statement. In the FROM clause, replace CDS View S4d430_Customer_With_Revenue with your new CDS View

Z##_Customer_With_Parameter. Supply the CDS View's input parameter with system field *sy-datum*.

a) See source code extract from the model solution.

4. Activate and test your program.

a) Perform this step as you did in previous exercises.

Executable Program S4D430_PARAM_S1

```
REPORT s4d430_param_s1.

TYPES ty_t_customers TYPE STANDARD TABLE OF
s4d430_customer_with_parameter
                        WITH NON-UNIQUE KEY id.

DATA gt_customers TYPE ty_t_customers.

* ALV Processing

DATA go_salv TYPE REF TO cl_salv_table.
DATA gx_excp TYPE REF TO cx_salv_error.

* Selection Screen

PARAMETERS pa_city TYPE s_city DEFAULT 'New York' LOWER CASE.

START-OF-SELECTION.

* Data Retrieval
*****

SELECT *
  FROM s4d430_customer_with_parameter( p_flightsbefore = @sy-datum )
 WHERE city = @pa_city
 ORDER BY city, name
 INTO TABLE @gt_customers.

* output
*****

cl_salv_table=>factory(
  IMPORTING
    r_salv_table = go_salv
  CHANGING
    t_table      = gt_customers
).

* Display
*-----*

go_salv->display( ).

CATCH cx_salv_error INTO gx_excp.      "
  MESSAGE gx_excp TYPE 'I'.
ENDTRY.
```

Task 3: System Field as Default Value for Input Parameter

Use an annotation to link your input parameter with system field *sy-datum*.

1. Edit your DDL Source `Z##_CUSTOMER_WITH_PARAMETER`. Use annotation `@Environment.systemField` for the input parameter to use the system date as the default value.
 - a) See source code extract from the model solution.
2. Activate the DDL Source and open the data preview for the new CDS View.

Why are you still prompted to enter a value for the input parameter? And why is there no default value for the parameter?

Annotation `@Environment.systemField` is only evaluated when the view is accessed from Open SQL.

- a) See the corresponding step in the previous exercise.
3. Return to your executable program `Z##_OPEN_SQL_PARAM` and perform a syntax check.
 - a) Choose *Syntax Check* from the toolbar.

Why do you suddenly get a syntax error that requires that you move the INTO clause to the very end of the SELECT statement?

Annotation `@Environment.systemField` is a feature available as off release 7.50. As soon as you use a 7.50 feature, your Open SQL statement has to fulfill the stricter syntax rules introduced with this release.

4. Adjust the SELECT statement according to the new syntax rules of release 7.50: Move the INTO clause to the very end of the statement.
 - a) See source code extract from the model solution.
5. Edit your program `Z##_OPEN_SQL_PARAM`. Copy the entire SELECT statement. In the copied SELECT statement, remove the parameter passing (brackets after the name of the CDS View).
 - a) See source code extract from the model solution.
6. Perform a syntax check to verify, that now a parameter passing is not needed anymore.
 - a) Choose *Syntax Check* from the toolbar.

DDL Source `S4D430_CUSTOMER_WITH_PARAMETER`

```
...
define view S4d430_Customer_With_PARAMETER
  with parameters
    @EndUserText.label: 'Consider flights up to this date'
    @Environment.systemField: #SYSTEM_DATE
    P_FlightsBefore : abap.dats
  as select
  from S4d430_Booking_And_Customer as c
...
```

Executable Program `S4D430_PARAM_S1`

```
* Option 2: Do not provide a value
* (possible if parameter is linked to a system field)
```

```
*****
*  SELECT * FROM s4d430_customer_with_PARAMETER
*          WHERE city = @pa_city
*          ORDER BY city, name
*          INTO TABLE @gt_customers.
```

Define a CDS View with Associations

Business Example

Template:

S4D430_BOOKING_AND_CUSTOMER (DDL Source)

Solution:

S4D430_ASSOCIATION (DDL Source)

Task 1: Define a CDS View With An Association

Create a copy of your CDS View that joins booking data and customer data. Replace the join with an (ad-hoc) association.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_ASSOCIATION, where ## stands for your group number).
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##ASSO).
3. Edit the new DDL Source. In the FROM clause, replace the inner join with an association to the CDS View with customer data.

Based on the data model, which would be the most appropriate cardinality for the relation from a flight booking to the flight customer?

4. Add the cardinality information to the association statement.
5. Perform a syntax check and activate your new DDL Source.

Task 2: Expose the Association

In the element list of your CDS view, comment out all elements that come from the associated view with customer data. Expose the complete association, instead.

1. In the element list of your CDS view, place characters "//" in front of all fields starting with "C.".



Hint:

You can comment out several lines by marking them and hitting keys *Strg* + *<*.

2. At the end of the element list, add the name of the association.

3. Apply the recommended naming rule for associations.



Note:

The rule recommends meaningful aliases for associations, starting with an underscore ("_").

4. Adjust the left hand side of the ON-clause. Refer to an element in the element list instead of a field of the data source.
5. Activate your new DDL Source.

Task 3: Use an Exposed Association in Data Preview

Open the data preview for your new CDS View and follow the association to the flight customer.

1. Open data preview for your new CDS View.
2. Choose a flight booking and display details of the flight customer associated to this booking.

Define a CDS View with Associations

Business Example

Template:

S4D430_BOOKING_AND_CUSTOMER (DDL Source)

Solution:

S4D430_ASSOCIATION (DDL Source)

Task 1: Define a CDS View With An Association

Create a copy of your CDS View that joins booking data and customer data. Replace the join with an (ad-hoc) association.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_ASSOCIATION, where ## stands for your group number).
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##ASSO).
 - a) See source code extract from the model solution.
3. Edit the new DDL Source. In the FROM clause, replace the inner join with an association to the CDS View with customer data.
 - a) Instead of `inner join <...> as C` write `association to <...> as C`.
 - b) See source code extract from the model solution.

Based on the data model, which would be the most appropriate cardinality for the relation from a flight booking to the flight customer?

As exactly one flight customer is assigned to every single flight booking, the most appropriate cardinality is 1..1.

4. Add the cardinality information to the association statement.
 - a) Add `[1..1]` immediately after keyword ASSOCIATION.
 - b) See source code extract from the model solution.
5. Perform a syntax check and activate your new DDL Source.
 - a) See the corresponding step in the previous exercise.

Task 2: Expose the Association

In the element list of your CDS view, comment out all elements that come from the associated view with customer data. Expose the complete association, instead.

1. In the element list of your CDS view, place characters “//” in front of all fields starting with “C.”.



Hint:

You can comment out several lines by marking them and hitting keys *Strg* + *<*.

- a) See source code extract from the model solution.
2. At the end of the element list, add the name of the association.
 - a) See source code extract from the model solution.
 3. Apply the recommended naming rule for associations.



Note:

The rule recommends meaningful aliases for associations, starting with an underscore (“_”).

- a) Replace `as C` with `as _Customer`.
 - b) Adjust all references to the alias.
 - c) See source code extract from the model solution.
4. Adjust the left hand side of the ON-clause. Refer to an element in the element list instead of a field of the data source.
 - a) Replace `b.customid` with `$procection.customid`.
 - b) See source code extract from the model solution.
 5. Activate your new DDL Source.
 - a) See the corresponding step in the previous exercise.

Task 3: Use an Exposed Association in Data Preview

Open the data preview for your new CDS View and follow the association to the flight customer.

1. Open data preview for your new CDS View.
 - a) See the corresponding step in the previous exercise.
2. Choose a flight booking and display details of the flight customer associated to this booking.
 - a) In the result list, right-click the booking for which you want to see customer details.
 - b) Choose *Follow Association*.
 - c) From the list of available associations, choose the one that leads to flight customer data.

DDL Source S4D430_ASSOCIATION

```
@AbapCatalog.sqlViewName: 'S4D430ASSO'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
```

```
@AccessControl.authorizationCheck: #CHECK
@EndUserText.label: 'Solution: Exposed Association'
define view S4d430_Association as
  select
    from S4D430_Booking as b
      association [1..1] to s4d430_Customer as _Customer
        on $projection.customid = _Customer.id
    {
      key b.carriid,
      key b.connid,
      key b.fldate,
      key b.bookid,
      b.customid,
      b.agencynum,
      b.amount,
      b.currency,
      b.days_ahead,
      _Customer // Make association public
        c.name,
        c.street,
        c.postcode,
        c.city,
        c.country
    }
}
```

Use a CDS View with Associations

Business Example

Template:

S4D430_CUSTOMER_WITH_PARAMETER (DDL Source)

S4D430_PATH_T1 (Executable Program)

Solution:

S4D430_CUSTOMER_WITH_PATH (DDL Source)

S4D430_PATH_S1 (Executable Program)

Task 1: Use an Exposed Association in Another CDS View

Create a copy of your CDS View that aggregates booking data per customer. Instead of reading from a CDS View with a join, read from the CDS view with an association.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_CUSTOMER_WITH_PATH, where ## stands for your group number).
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##CUSTASSO).
3. Edit the new DDL Source. Replace the CDS View in the FROM clause with your CDS View Z##_Association, which contains an exposed association. If you did not complete the respective exercise, you can read from CDS View S4d430_Association, instead.
4. Perform a syntax check.

Why do you get syntax errors?

How can you read the customer-related information?

5. Use path expressions to read the flight customer data from the associated data source.
6. Perform a syntax check and activate your new DDL Source.

Task 2: Use an Exposed Association in Open SQL

Access your CDS View with exposed association in Open SQL. Use path expressions to follow the association.

1. Create a copy of executable program S4D430_PATH_T1 (suggested name: Z##R_PATH, where ## is your group number).
2. Change the SELECT statement. Replace the CDS View in the FROM clause with your CDS View Z##_Association. If you did not complete the respective exercise, you can use CDS View S4d430_Association, instead.



Note:

Do not adjust the definition of local data type *ty_s_customer*!

3. Perform a syntax check for your executable error.

Why do you get syntax errors?

How can you access the customer-related information, now?

4. In your SELECT statement, use a path expression for all flight customer-related fields. Do not forget fields in the WHERE clause and ORDER BY clause.



Hint:

Path expressions in Open SQL require a backslash (character "\") in front of each association name and dash (character "-") is used as component selector.

5. Activate and test your program.

Use a CDS View with Associations

Business Example

Template:

S4D430_CUSTOMER_WITH_PARAMETER (DDL Source)

S4D430_PATH_T1 (Executable Program)

Solution:

S4D430_CUSTOMER_WITH_PATH (DDL Source)

S4D430_PATH_S1 (Executable Program)

Task 1: Use an Exposed Association in Another CDS View

Create a copy of your CDS View that aggregates booking data per customer. Instead of reading from a CDS View with a join, read from the CDS view with an association.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: Z##_CUSTOMER_WITH_PATH, where ## stands for your group number).
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: Z##CUSTASSO).
 - a) See source code extract from the model solution.
3. Edit the new DDL Source. Replace the CDS View in the FROM clause with your CDS View Z##_Association, which contains an exposed association. If you did not complete the respective exercise, you can read from CDS View S4d430_Association, instead.
 - a) See source code extract from the model solution.
4. Perform a syntax check.
 - a) See the corresponding step in previous exercises.

Why do you get syntax errors?

The customer-related fields (NAME, STREET, POSTCODE, CITY, COUNTRY) are not contained in the new data source.

How can you read the customer-related information?

Use the association in path expressions.

5. Use path expressions to read the flight customer data from the associated data source.

- a) Use path expressions like `c._Customer.name`.
 - b) See source code extract from the model solution.
6. Perform a syntax check and activate your new DDL Source.
- a) See the corresponding step in previous exercises.

DDL Source S4D430_CUSTOMER_WITH_PATH

```
@AbapCatalog.sqlViewName: 'S4D430_CUSTPATH'
@AbapCatalog.compiler.compareFilter: true
@AbapCatalog.preserveKey: true
@AccessControl.authorizationCheck: #CHECK
@endUserText.label: 'Solution: Path Expressions in CDS DDL'
define view S4d430_Customer_With_Path
  with parameters
    @EndUserText.label: 'Consider flights up to this date'
    @Environment.systemField: #SYSTEM_DATE
    P_FlightsBefore : abap.dats
  as select
    from S4d430_Association as c
  {
    key c.customid as id,
    c._Customer.name,
    c._Customer.street,
    c._Customer.postcode,
    c._Customer.city,
    c._Customer.country,
    sum(c.amount) as amount @<Semantics.amount.currencyCode:
'currency',
    c.currency,
    avg(c.days_ahead) as days_ahead
  }
  where c.fldate < $parameters.P_FlightsBefore
  group by
    c.customid,
    c._Customer.name,
    c._Customer.street,
    c._Customer.postcode,
    c._Customer.city,
    c._Customer.country,
    c.currency
```

Task 2: Use an Exposed Association in Open SQL

Access your CDS View with exposed association in Open SQL. Use path expressions to follow the association.

1. Create a copy of executable program S4D430_PATH_T1 (suggested name: Z##R_PATH, where ## is your group number).
 - a) See the corresponding step in previous exercises.
2. Change the SELECT statement. Replace the CDS View in the FROM clause with your CDS View Z##_Association. If you did not complete the respective exercise, you can use CDS View S4d430_Association, instead.



Note:

Do not adjust the definition of local data type `ty_s_customer`!

- a) See source code extract from the model solution.

3. Perform a syntax check for your executable error.
 - a) See the corresponding step in previous exercises.

Why do you get syntax errors?

The customer-related fields (NAME, STREET, POSTCODE, CITY, COUNTRY) are not contained in the new CDS View, directly.

How can you access the customer-related information, now?

Use the association in path expressions.

4. In your SELECT statement, use a path expression for all flight customer-related fields. Do not forget fields in the WHERE clause and ORDER BY clause.



Hint:

Path expressions in Open SQL require a backslash (character "\") in front of each association name and dash (character "-") is used as component selector.

- a) Your path expressions should look like _Customer-name, _Customer-street, etc.
 - b) See source code extract from the model solution.
5. Activate and test your program.
 - a) Perform this step as you did in previous exercises.

Executable Program S4D430_PATH_S1

```
REPORT s4d430_path_s1.

TYPES: BEGIN OF ty_s_customer,
        id          TYPE s4d430_customer_with_parameter-id,
        name        TYPE s4d430_customer_with_parameter-name,
        street       TYPE s4d430_customer_with_parameter-street,
        postcode     TYPE s4d430_customer_with_parameter-postcode,
        city         TYPE s4d430_customer_with_parameter-city,
        country      TYPE s4d430_customer_with_parameter-country,
        amount       TYPE s4d430_customer_with_parameter-amount,
        currency     TYPE s4d430_customer_with_parameter-currency,
        days_ahead   TYPE s4d430_customer_with_parameter-days_ahead,
      END OF ty_s_customer.

TYPES ty_t_customers TYPE STANDARD TABLE OF ty_s_customer
      WITH NON-UNIQUE KEY id.

DATA gt_customers TYPE ty_t_customers.

* ALV Processing

DATA go_salv TYPE REF TO cl_salv_table.
DATA gx_excp TYPE REF TO cx_salv_error.
```

```

* Selection Screen

PARAMETERS pa_city TYPE s_city DEFAULT 'New York' LOWER CASE.

START-OF-SELECTION.

* Data Retrieval
*****

SELECT
  FROM s4d430_association
  FIELDS customid as id,
         \_customer-name,
         \_customer-street,
         \_customer-postcode,
         \_customer-city,
         \_customer-country,
         sum( amount ) as amount,
         currency,
         avg( days_ahead ) as days_ahead
  WHERE \_customer-city = @pa_city
        and fdate < @sy-datum
  group by customid,
         \_customer-name,
         \_customer-street,
         \_customer-postcode,
         \_customer-city,
         \_customer-country,
         currency
  ORDER BY \_customer-city, \_customer-name
INTO TABLE @gt_customers.

* output
*****

TRY.

  cl_salv_table=>factory(
    IMPORTING
      r_salv_table = go_salv
    CHANGING
      t_table      = gt_customers
  ).

* Display
*-----*

  go_salv->display( ).

  CATCH cx_salv_error INTO gx_excp.      "
    MESSAGE gx_excp TYPE 'I'.
  ENDTRY.

```

Enhance a CDS View

Business Example

Template:

S4D430_ASSOCIATION (DDL Source)

Solution:

S4D430_EXTENDED (DDL Source)

S4D430_EXTENSION (DDL Source)

Task 1: Create a Copy of the Template CDS View

Create a copy of your CDS View that exposes an association. If you did not complete the respective exercise, you can copy from CDS View *S4d430_Association*.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: *Z##_EXTENDED*, where *##* stands for your group number).
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: *Z###EXTENDED*).
3. Perform a syntax check and activate your new DDL Source.

Task 2: Define a CDS Extend View

Define a CDS Extend View for your CDS View *Z##_Extended*. Add the counter number and the flight class to the element list. Add an association to table SPFLI and expose it in the field list.

1. In your own Package, create a new DDL Source (suggested name: *Z##_Extension*, where *##* stands for your group number). Choose template *extendView*.
2. In the new DDL Source, provide a suitable name for the SQL View Append (suggested name: *Z###EXTENSION*).
3. After keyword *EXTEND VIEW*, enter the name of the CDS View you want to extend (*Z##_Extended*).
4. In the element list, add fields *CLASS* and *COUNTER* from the original data source.



Hint:

Use the source code completion capabilities of the editor to choose from the available fields.

5. After the name of the Extend View in the *WITH* addition, add an association to table *SPFLI*. Provide a suitable name for the association (suggested name: *_connection*) and specify the join condition.
6. Expose the entire new association in the element list.

7. Perform a syntax check and activate your new DDL Source.

Task 3: Check the View Extension

Open the definition of the extended view and analyze the extension. Open the extended view in Data Preview and make sure that the additional fields and associations are displayed.

1. Open the definition of your CDS View `Z##_Extended`.

How can you see that this view has been extended?

2. Navigate from the extended view into the definition of the CDS Extend View.
3. Return to the extended view and open it with data preview.
4. Confirm that fields `CLASS` and `COUNTER` are there and that they are filled with data.
5. Confirm that association `_Connection` is there and that you can follow it in data preview.

Enhance a CDS View

Business Example

Template:

S4D430_ASSOCIATION (DDL Source)

Solution:

S4D430_EXTENDED (DDL Source)

S4D430_EXTENSION (DDL Source)

Task 1: Create a Copy of the Template CDS View

Create a copy of your CDS View that exposes an association. If you did not complete the respective exercise, you can copy from CDS View *S4d430_Association*.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: *Z##_EXTENDED*, where *##* stands for your group number).
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: *Z##EXTENDED*).
 - a) See source code extract from the model solution.
3. Perform a syntax check and activate your new DDL Source.
 - a) See the corresponding step in previous exercises.

Task 2: Define a CDS Extend View

Define a CDS Extend View for your CDS View *Z##_Extended*. Add the counter number and the flight class to the element list. Add an association to table *SPFLI* and expose it in the field list.

1. In your own Package, create a new DDL Source (suggested name: *Z##_Extension*, where *##* stands for your group number). Choose template *extendView*.
 - a) See the corresponding steps in previous exercises.
2. In the new DDL Source, provide a suitable name for the SQL View Append (suggested name: *Z##EXTENSION*).
 - a) See source code extract from the model solution.
3. After keyword *EXTEND VIEW*, enter the name of the CDS View you want to extend (*Z##_Extended*).
 - a) See source code extract from the model solution.
4. In the element list, add fields *CLASS* and *COUNTER* from the original data source.



Hint:

Use the source code completion capabilities of the editor to choose from the available fields.

- a) See source code extract from the model solution.
- 5. After the name of the Extend View in the WITH addition, add an association to table SPFLI. Provide a suitable name for the association (suggested name: `_connection`) and specify the join condition.
 - a) See source code extract from the model solution.
- 6. Expose the entire new association in the element list.
 - a) See source code extract from the model solution.
- 7. Perform a syntax check and activate your new DDL Source.
 - a) See the corresponding step in the previous exercise.

Task 3: Check the View Extension

Open the definition of the extended view and analyze the extension. Open the extended view in Data Preview and make sure that the additional fields and associations are displayed.

- 1. Open the definition of your CDS View `Z##_Extended`.
 - a) See the corresponding step in previous exercises.

How can you see that this view has been extended?

There is a spiral-like (or snail-like) symbol next to the code line that contains the DEFINE VIEW statement.

- 2. Navigate from the extended view into the definition of the CDS Extend View.
 - a) Click on the spiral-like (or snail-like) symbol next to the code line that contains the DEFINE VIEW statement.
 - b) On the dialog box that appears, click on the name of the CDS Extend View.
- 3. Return to the extended view and open it with data preview.
 - a) Right-click anywhere in the DDL Source and choose *Open With* → *Data Preview*.
- 4. Confirm that fields CLASS and COUNTER are there and that they are filled with data.
 - a) You should find the new fields on the far right of the data preview display.



Hint:

Not many bookings have a value in column COUNTER. You will need to scroll down or you might even have to apply a filter to see entries with a non-initial value in this column.

- 5. Confirm that association `_Connection` is there and that you can follow it in data preview.
 - a) Right-click on any line in the preview and choose *Follow Association*.
 - b) In the dialog window, click on association `_Connection`.

DDL Source S4D430_EXTENSION

```
@AbapCatalog.sqlViewAppendName: 'S4D430_EXTVIEW'
@EndUserText.label: 'Solution: View Extension'
extend view S4d430_Extended
  with S4D430_Extension
  association[1] to spfli as _connection
    on b.carriid = _connection.carriid
    and b.connid = _connection.connid
  {
    b.class,
    b.counter,
    _connection
  }
```

Create and Use an Access Control (DCL Source)

Business Example

Template:

S4D430_ASSOCIATION (DDL Source)

Solution:

S4D430_AUTHORIZATION (DDL Source)

S4D430_ACCESS_CONTROL (DCL Source)

Task 1: Create a Copy of the Template CDS View

Create a copy of your CDS View that exposes an association. If you did not complete the respective exercise, you can copy from DDL Source *S4D430_ASSOCIATION*.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: *Z##_AUTHORIZATIONS*, where *##* stands for your group number).
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: *Z##AUTH*).
3. Perform a syntax check and activate your new DDL Source.

Task 2: Define A CDS Role

Define a CDS Role for your CDS View *Z##_Authorization*. Make sure only bookings of those airlines are selected for which the user got reading authorization.

1. In your own Package, create a new DCL Source (suggested name: *Z##_Access_Control*, where *##* stands for your group number). Choose template *DefineMappingRole*.
2. Edit the new DCL Source. After keyword *DEFINE ROLE*, enter a name for the CDS Role.



Hint:

It is recommended to use the same name for the DCL Source and the CDS Role (apart from capitalization). Therefore, the recommended name for the rule is *Z##_Access_Control*.

3. After keyword *GRANT SELECT ON*, enter the name of the CDS View you want to link with authorization rules (*Z##_Authority*).
4. After keyword *WHERE*, enter the name of column *CARRID* in brackets.
5. Provide input for function *pfcg_auth()*. First, enter the name of the authorization object, then the name of the authorization field that should be supplied with view field *CARRID*. The final argument for function *pfcg_auth()* should be condition *ACTVT = '03'*.



Hint:
The correct authorization object is S_CARRID.

6. Perform a syntax check and activate your new DCL Source.

Task 3: Check the Authorization

Open the definition of the view for which you just defined a CDS Role. Open the Data Preview and make sure that only the data is displayed for which your user is authorized.

1. Open the definition of your CDS View Z##_Authorization.

Where can you see that there is a CDS role assigned to this CDS View?

2. Open the CDS View with data preview.
3. Confirm that the result does not contain bookings for which your user does not have display authorization.



Hint:
Your user should not have display authorization for carrier UA.

4. Confirm that association *_Connection* is there and that you can follow it in data preview.

Create and Use an Access Control (DCL Source)

Business Example

Template:

S4D430_ASSOCIATION (DDL Source)

Solution:

S4D430_AUTHORIZATION (DDL Source)

S4D430_ACCESS_CONTROL (DCL Source)

Task 1: Create a Copy of the Template CDS View

Create a copy of your CDS View that exposes an association. If you did not complete the respective exercise, you can copy from DDL Source *S4D430_ASSOCIATION*.

1. In your own package, create a copy of your own DDL Source or of the template (suggested name: *Z##_AUTHORIZATIONS*, where *##* stands for your group number).
 - a) See the corresponding step in previous exercises.
2. In the new DDL Source, provide a suitable name for the SQL View (suggested name: *Z##AUTH*).
 - a) See source code extract from the model solution.
3. Perform a syntax check and activate your new DDL Source.
 - a) See the corresponding step in previous exercises.

Task 2: Define A CDS Role

Define a CDS Role for your CDS View *Z##_Authorization*. Make sure only bookings of those airlines are selected for which the user got reading authorization.

1. In your own Package, create a new DCL Source (suggested name: *Z##_Access_Control*, where *##* stands for your group number). Choose template *DefineMappingRole*.
 - a) In the *ABAP* perspective of *Eclipse*, right-click on your project and choose *New* → *Other ABAP Repository Object*.
 - b) On the next window, choose *Core Data Services* → *Access Control*.
 - c) Enter the name of your package, a name, and a short description for the new object and choose *Next*.
 - d) If the tool asks for a template, select *DefineMappingRole* and choose *Next*.
 - e) Assign the object to your transport request and choose *Finish*.
2. Edit the new DCL Source. After keyword *DEFINE ROLE*, enter a name for the CDS Role.



Hint:

It is recommended to use the same name for the DCL Source and the CDS Role (apart from capitalization). Therefore, the recommended name for the rule is `Z##_Access_Control`.

3. After keyword `GRANT SELECT ON`, enter the name of the CDS View you want to link with authorization rules (`Z##_Authority`).
 - a) See source code extract from the model solution.
4. After keyword `WHERE`, enter the name of column `CARRID` in brackets.
 - a) See source code extract from the model solution.
5. Provide input for function `pfcg_auth()`. First, enter the name of the authorization object, then the name of the authorization field that should be supplied with view field `CARRID`. The final argument for function `pfcg_auth()` should be condition `ACTVT = '03'`.



Hint:

The correct authorization object is `S_CARRID`.

- a) See source code extract from the model solution.
6. Perform a syntax check and activate your new DCL Source.
 - a) See the corresponding step in previous exercises.

Task 3: Check the Authorization

Open the definition of the view for which you just defined a CDS Role. Open the Data Preview and make sure that only the data is displayed for which your user is authorized.

1. Open the definition of your CDS View `Z##_Authorization`.
 - a) See the corresponding step in previous exercises.

Where can you see that there is a CDS role assigned to this CDS View?

On the *problems* tab below the editor, there is a corresponding message in node *Infos*.

2. Open the CDS View with data preview.
 - a) Right-click anywhere in the DDL Source and choose *Open With → Data Preview*.
3. Confirm that the result does not contain bookings for which your user does not have display authorization.



Hint:

Your user should not have display authorization for carrier `UA`.

- a) Sort the result by field `CARRID` and scroll down.
- b) Alternatively, you can right-click on column `CARRID` and choose *Distinct Values for [carrid]*.

4. Confirm that association *_Connection* is there and that you can follow it in data preview.
 - a) Right-click on any line in the preview and choose *Follow Association*.
 - b) In the dialog window, click on associaton *_Connection*.

DCL Source S4D430_ACCESS_CONTROL

```
@EndUserText.label: 'Solution: Role for View S4d430_Authorization'
@MappingRole: true
define role S4d430_Access_Control
{ grant select
  on S4d430_authorization
  where ( CARRID ) = aspect pfcg_auth( S_CARRID,
                                      CARRID,
                                      ACTVT = '03'
                                      );
}
```

Define and Use a CDS Table Function

Business Example

Template:

CL_S4D430_TABLE_FUNC_T1 (Global Class)
S4D430_TABLE_FUNC_T1 (Executable Program)

Solution:

CL_S4D430_TABLE_FUNC_S1 (Global Class)
S4D430_Customer_Fuzzy (DDL Source)
S4D430_TABLE_FUNC_S1 (Executable Program)

Task 1: Copy Template Class with Dummy Method

Create a copy of template class CL_S4D430_TABLE_FUNC_T1 and activate it.

1. In your own Package, create a copy of global class CL_S4D430_TABLE_FUNC_T1 (suggested name: ZCL_##_TABLE_FUNC, where ## stands for your group number).



Hint:

If you want to perform this step in ADT, right click the global class and choose *Duplicate*.

2. Activate the new global class.

Task 2: Define a CDS Table Function

Define a CDS Table Function that imports a customer name and returns a list of customer data.

1. In your own Package, create a new DDL Source (suggested name: Z##_Customer_Fuzzy, where ## stands for your group number). Choose template *defineTableFunctionWithParameters*.
2. In the new DDL Source, define an input parameter of type S_CUSTNAME (suggested name: NAME_IN).
3. Within the curly brackets after keyword RETURNS, enter a list of customer data that should be returned by the table function. This list should include the client, the customer id, the customer name, the city, and the country. Make sure to use delimiter “;” after each return field (including the last!).



Hint:

You can look up appropriate data elements in the definition of database table SCUSTOM.

4. After keyword `IMPLEMENTED BY METHOD`, specify the name of your global class and the name of the dummy method in this class.



Hint:

You can use the source code completion capabilities of the editor to enter this information.

5. Perform a syntax check and activate your new DDL Source.

Task 3: Implement the AMDP Method for the CDS Table Function

In your global class, define and implement an AMDP method by uncommenting the method implementation. Connect this AMDP Method to your CDS Table Function.

1. Edit your global class. Go to the implementation of method `customer_fuzzy`. Remove the full stop after the method name and uncomment the additions `BY DATABASE PROCEDURE ... USING SCUSTOM..`
2. Perform a syntax check.

What does the syntax error say?

3. Adjust the method definition. Do not add parameters, directly. Use addition `FOR TABLE FUNCTION` instead and add the name of your CDS Table function. The method then takes over the import parameter(s) and the structure of the returning parameter from the CDS Table Function.
4. Perform another syntax check.

What does the next syntax error say?

5. In the global section of the class definition, add interface `IF_AMD_MARKER_HDB`.
6. Complete the implementation by uncommenting the SQL script coding in the AMDP method.
7. Activate your global class.

Task 4: Access CDS Table Function with Open SQL

Create a copy of executable program `S4D430_TABLE_FUNC_T1`. In the `FROM` clause of the `SELECT` statement, replace transparent table `SCUSTOM` with your CDS Table Function.

Optionally, add coding that checks whether the current database supports CDS Table functions; and if it doesn't, issue an error message.

1. Create a copy of executable program S4D430_TABLE_FUNC_T1 (suggested name of the copy: Z##_TABLE_FUNC, where ## stands for your group number).



Hint:

If you want to perform this step in *ABAP Development Tools*, choose *Duplicate* from the context menu.

2. In the From clause of the SELECT statement, remove transparent table SCUSTOM and replace it with your CDS Table Function Z##_Customer_Fuzzy.



Note:

Don't forget to supply an actual value for the table function's input parameter.

3. Execute a syntax check.

Why do you get a warning from syntax check?

4. Optional: Before the select statement, implement a call of static method *use_features* of global class CL_ABAP_DBFEATURES. If table functions are not supported in this system, issue an error message, e.g. message 030 of message class S4D430.
5. Activate and test your program.

Define and Use a CDS Table Function

Business Example

Template:

CL_S4D430_TABLE_FUNC_T1 (Global Class)
S4D430_TABLE_FUNC_T1 (Executable Program)

Solution:

CL_S4D430_TABLE_FUNC_S1 (Global Class)
S4D430_Customer_Fuzzy (DDL Source)
S4D430_TABLE_FUNC_S1 (Executable Program)

Task 1: Copy Template Class with Dummy Method

Create a copy of template class CL_S4D430_TABLE_FUNC_T1 and activate it.

1. In your own Package, create a copy of global class CL_S4D430_TABLE_FUNC_T1 (suggested name: ZCL_##_TABLE_FUNC, where ## stands for your group number).



Hint:

If you want to perform this step in ADT, right click the global class and choose *Duplicate*.

- a) See the corresponding step in previous exercises.
2. Activate the new global class.
 - a) See the corresponding step in previous exercises.

Task 2: Define a CDS Table Function

Define a CDS Table Function that imports a customer name and returns a list of customer data.

1. In your own Package, create a new DDL Source (suggested name: Z##_Customer_Fuzzy, where ## stands for your group number). Choose template *defineTableFunctionWithParameters*.
 - a) See the corresponding steps in the previous exercise.
2. In the new DDL Source, define an input parameter of type S_CUSTNAME (suggested name: NAME_IN).
 - a) See source code extract from the model solution.
3. Within the curly brackets after keyword RETURNS, enter a list of customer data that should be returned by the table function. This list should include the client, the customer id, the customer name, the city, and the country. Make sure to use delimiter “;” after each return field (including the last!).



Hint:

You can look up appropriate data elements in the definition of database table SCUSTOM.

- a) See source code extract from the model solution.
- 4. After keyword **IMPLEMENTED BY METHOD**, specify the name of your global class and the name of the dummy method in this class.



Hint:

You can use the source code completion capabilities of the editor to enter this information.

- a) See source code extract from the model solution.
- 5. Perform a syntax check and activate your new DDL Source.
 - a) See the corresponding step in previous exercises.

DDL Source S4D430_CUSTOMER_FUZZY

```
@EndUserText.label: 'Solution: Table Function'
define table function S4D430_Customer_Fuzzy
with parameters name_in : s_custname
returns {
    mandt      : mandt;
    id         : s_customer;
    name       : s_custname;
    city       : s_city;
    country    : s_country;}
```

```
implemented by method CL_S4D430_TABLE_FUNC_S1=>CUSTOMER_FUZZY;
```

Task 3: Implement the AMDP Method for the CDS Table Function

In your global class, define and implement an AMDP method by uncommenting the method implementation. Connect this AMDP Method to your CDS Table Function.

1. Edit your global class. Go to the implementation of method *customer_fuzzy*. Remove the full stop after the method name and uncomment the additions **BY DATABASE PROCEDURE ... USING SCUSTOM..**
 - a) See source code extract from the model solution.
2. Perform a syntax check.
 - a) See relevant steps in previous exercises.

What does the syntax error say?

The method ... contains a database function, but there is no RETURNING parameter defined.

3. Adjust the method definition. Do not add parameters, directly. Use addition **FOR TABLE FUNCTION** instead and add the name of your CDS Table function. The method then takes over the import parameter(s) and the structure of the returning parameter from the CDS Table Function.

- a) See source code extract from the model solution.
- 4. Perform another syntax check.
 - a) See relevant steps in previous exercises.

What does the next syntax error say?

Database procedures managed by ABAP can only be defined in classes that implement the marker interface "IF_AMDP_MARKER_HDB".

- 5. In the global section of the class definition, add interface IF_AMD_MARKER_HDB.
 - a) See source code extract from the model solution.
- 6. Complete the implementation by uncommenting the SQL script coding in the AMDP method.
 - a) See source code extract from the model solution.
- 7. Activate your global class.
 - a) See the relevant steps in previous exercises.

Global Class CL_S4D430_TABLE_FUNC_S1

```

CLASS cl_s4d430_table_func_s1 DEFINITION PUBLIC FINAL CREATE PUBLIC .

    PUBLIC SECTION.

        INTERFACES if_amdp_marker_hdb.

        CLASS-METHODS customer_fuzzy FOR TABLE FUNCTION
        s4d430_customer_fuzzy.

ENDCLASS.


CLASS cl_s4d430_table_func_s1 IMPLEMENTATION.

    METHOD customer_fuzzy BY DATABASE FUNCTION
                        FOR HDB LANGUAGE SQLSCRIPT
                        OPTIONS READ-ONLY
                        USING scustom.

        RETURN SELECT mandt, id, name, city, country
               FROM scustom
               WHERE contains( name, :name_in, fuzzy )
               ORDER BY score( ) desc;

    ENDMETHOD.

ENDCLASS.

```

Task 4: Access CDS Table Function with Open SQL

Create a copy of executable program S4D430_TABLE_FUNC_T1. In the FROM clause of the SELECT statement, replace transparent table SCUSTOM with your CDS Table Function. Optionally, add coding that checks whether the current database supports CDS Table functions; and if it doesn't, issue an error message.

- 1. Create a copy of executable program S4D430_TABLE_FUNC_T1 (suggested name of the copy: Z##_TABLE_FUNC, where ## stands for your group number).



Hint:

If you want to perform this step in *ABAP Development Tools*, choose *Duplicate* from the context menu.

- a) See relevant steps in previous exercises.
2. In the From clause of the SELECT statement, remove transparent table SCUSTOM and replace it with your CDS Table Function Z##_Customer_Fuzzy.



Note:

Don't forget to supply an actual value for the table function's input parameter.

- a) See source code extract from the model solution.
3. Execute a syntax check.

Why do you get a warning from syntax check?

Because table functions are not supported by all database systems, you need to check before this SELECT statement, that it is supported in the current system.

- a) See relevant steps in previous exercises.
4. Optional: Before the select statement, implement a call of static method *use_features* of global class CL_ABAP_DBFEATURES. If table functions are not supported in this system, issue an error message, e.g. message 030 of message class S4D430.
- a) See source code extract from the model solution.
5. Activate and test your program.
- a) See relevant steps in previous exercises.

Executable Program S4D430_TABLE_FUNC_S1

```
REPORT s4d430_table_func_s1.

TYPES: BEGIN OF ty_s_customer,
        id          TYPE scustom-id,
        name        TYPE scustom-name,
        city        TYPE scustom-city,
        country     TYPE scustom-country,
      END OF ty_s_customer.

TYPES ty_t_customers TYPE STANDARD TABLE OF ty_s_customer
      WITH NON-UNIQUE KEY id.

DATA gt_customers TYPE ty_t_customers.

* ALV Processing

DATA go_salv TYPE REF TO cl_salv_table.
DATA gx_excp TYPE REF TO cx_salv_error.
```

```

* Selection Screen

PARAMETERS pa_nam TYPE s_custname DEFAULT 'Schwarz' LOWER CASE.

* Check db feature

DATA gt_feature_set TYPE cl_abap_dbfeatures=>features_set_t.

START-OF-SELECTION.

* Optional: Check if table functions can be used on this DB
*****
INSERT cl_abap_dbfeatures=>amdp_table_function
      INTO TABLE gt_feature_set.

IF cl_abap_dbfeatures=>use_features( gt_feature_set )
  = abap_false.
  MESSAGE e030(S4D430).
ENDIF.

* Alternative with VALUE expression (Rel. 7.40)

*IF cl_abap_dbfeatures=>use_features(
*   VALUE #( ( cl_abap_dbfeatures=>amdp_table_function ) )
*   ) = abap_false.
* MESSAGE e030(S4D430).
*ENDIF.

* Data Retrieval
*****

SELECT
  FROM s4d430_table_function( name_in = @pa_nam )
  FIELDS id,
         name,
         city,
         country
  ORDER BY country, city, name
INTO TABLE @gt_customers.

* output
*****
TRY.

  cl_salv_table=>factory(
    IMPORTING
      r_salv_table = go_salv
    CHANGING
      t_table      = gt_customers
  ).

* Display
*-----*

  go_salv->display( ).

  CATCH cx_salv_error INTO gx_excp.    "
    MESSAGE gx_excp TYPE 'I'.
  ENDTRY.

```